

第二部分 · 项目面经

54 道项目题 · 两段口述回答 → 对应八股 → 实现证据与追问

项目全景与回答主线

网关并发与 Linux I/O

帧协议、命令可靠性与 MQTT

数据存储、热加载与运维

节点采集、任务与串口

时钟、低功耗与故障定位

FreeRTOS 移植与启动验收

项目全景与回答主线

P001 项目问答

请用两分钟介绍这个网关与节点项目。

面试时这样回答

这是一个由 Linux 网关和 STM32 节点组成的采集与控制系统。节点用 FreeRTOS 管理传感器采集、命令处理和串口发送；网关接收节点数据，转换后上传 MQTT 并写入 SQLite，同时提供 HTTP 查询。反方向，云端命令通过网关下发到节点，节点执行后把结果返回。

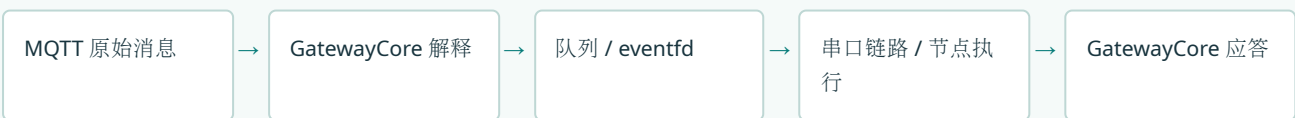
网关采用多个 I/O 事件循环加一个共享业务线程：串口 epoll、MQTT 网络循环和 Boost.Beast/Asio HTTP 分别管理通信，GatewayCore 顺序处理设备业务，SQLite 查询和写入交给各自执行线程。控制链使用带会话号的 SR 协议，区分“已接收”和“已执行”，处理丢帧、重复和超时。

先记住这两条数据路径

遥测上行



控制下行



回到第一部分

T013 · 进程、线程和 FreeRTOS 任务有什么区别？

T016 · 阻塞、非阻塞、同步、异步如何区分？

T036 · 如何设计一条能抗分包和噪声的串口帧？

T084 · FreeRTOS 的任务状态和调度链路如何理解？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

继续展开时沿两条路径讲

上行是节点采样→发送队列→串口 Reactor 完成帧→GatewayCore 转换→MQTT/数据库；下行是 MQTT 原始消息→GatewayCore 解析→串口 Reactor 管链路→节点执行→GatewayCore 生成应答。先讲消息交接与状态所有者，再展开 DMA、重试和存储。

网关 GatewayApp.cpp:113 · 节点 app.c:13 · 网关 edge_proto.h:110 · 网关 TelemetryPipeline.cpp:45 · 网关 GatewayCore.cpp:118

继续追问

这个项目最主要解决什么工程问题？

让网络、串口和磁盘等异步来源在有限资源内协作：事件循环不做数据库查询或业务格式化，同一设备的业务逻辑有明确执行线程，链路重试不等于重新执行业务。仍需分别说明过载、断电和跨会话恢复边界。

当前实现边界

这段介绍描述系统能力，不自动证明每个模块都由同一个人完成。个人贡献应按实际参与范围说明；前端页面不归为本次架构改造的个人开发内容。

P002 项目问答

一条温湿度数据从节点到云端和数据库，具体怎么走？

面试时这样回答

采样任务先通过传感器服务读取数据，并拿到温湿度是否有效的标志。有效值转换成协议规定的整数单位，编码成一条完整帧，再按值复制进发送队列，由发送任务统一写串口。这样采样任务不直接和其他任务抢发送口。

网关串口 Reactor 增量拼帧并检查 CRC，将完整遥测帧和接收时间交给 GatewayCore。业务线程解码温湿度、生成 MQTT 消息并提交数据库写入；HTTP 的历史查询另交读执行线程，SQL 不在串口、HTTP 或业务线程直接执行。

回到第一部分

T006 · 结构体为什么不能直接当作串口协议发送？

T054 · SQLite 的连接、读写并发与线程模式怎么理解？

T055 · 批量写入为什么能提速，怎样控制额外延迟？

T087 · 队列为什么复制数据，传指针又有什么代价？

T114 · 定点编码、单位、有效性与采样时间如何设计？

实现依据与展开

数值与时间的具体依据

温湿度在线上按十分之一单位编码，节点保留合法小数；网关时间戳在串口收帧时取得，不是业务出队时间或节点精确采样时刻。跨线程帧拥有自己的数据，不能借用接收回调的临时引用；MQTT 发布和数据库写入是两条没有共同事务的支路。

节点 report_task.c:55 · 节点 sensor_hub.c:19 · 节点 dht11.c:51 · 节点 tx_service.c:22 · 网关 NodeLink.cpp:92 · 网关 GatewayApp.cpp:113 · 网关 TelemetryPipeline.cpp:45 · 网关 GatewayCore.cpp:118

继续追问

上报成功是否代表数据库已经写成功？

不能。节点入队、网关收帧、业务接纳、MQTT 库接受发布、写库入队和事务提交是不同阶段；线程解耦不自动增加端到端可靠确认。

当前实现边界

遥测入口或写库队列满时仍可能丢数据；传感器无效时不会把失败伪装成正常读数。拆分收益尚不能表述为已测得的延迟或吞吐提升。

P003 项目问答

一条云端查询命令怎样形成完整闭环？

面试时这样回答

MQTT 网络回调复制并投递 topic/payload，GatewayCore 在业务线程调用 translateCommand，再向串口命令队列提交内部命令并用 eventfd 唤醒。串口 Reactor 在会话就绪、窗口有空间时分配链路序号、组帧并发送。节点先保存命令并确认接收，再按序执行业务。

节点把结果存入缓存后尝试发回。串口 Reactor 处理 SR 状态与结果确认，把已完成命令事件交给 GatewayCore 映射数据、返回码和云端应答。网络循环负责实际 MQTT 收发；各线程都不为等待节点回应而阻塞。

回到第一部分

T029 · eventfd 为什么配一个队列，计数是否等于命令条数？

T039 · 传输确认、按序交付与业务完成为什么不是同一状态？

T040 · 会话号与序号为什么缺一不可？

T041 · 怎样设计 RTO、总截止时间与有限重试？

T049 · MQTT QoS 0、1、2 到底保证什么？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

读代码时的主线

createMqttClient 消息回调→GatewayCore 任务→translateCommand→下行队列/eventfd→CommandTracker/NodeLink; 节点 vCmdTask→edge_sr_node_on_frame→execute→传感器服务。回程由 tracker_.onFrame/applyTrackerActions 分离协议发送动作与完成事件, 再由 GatewayCore::handleFinished 处理应用结果。

网关 MqttClient.cpp:21 · 网关 CommandTranslator.cpp:57 · 网关 GatewayApp.cpp:113 · 网关 CommandTracker.cpp:78 · 节点 cmd_service.c:77 · 网关 edge_sr.c:62 · 网关 GatewayCore.cpp:118

继续追问

这里哪一个 **ACK** 表示执行成功?

SR_RECEIVED 只表示节点保存请求。业务结果的返回码与数据才描述执行结局; SR_RESULT_ACK 主要用于节点释放结果缓存。收到重复结果仍须确认, 但不能重复完成上层请求。

当前实现边界

云端命令目前没有跨重连、跨进程重启的持久业务 ID; 链路闭环不能直接外推成整个云端业务恰好一次。

P004 项目问答

这种架构属于哪种 **Reactor** 模式, 为什么共用业务线程?

面试时这样回答

整个网关仍在一个进程内, 串口、MQTT 和 HTTP 各有 I/O 事件循环, 因此属于多事件循环、多线程架构。它按通信通道分工, 各循环把应用事件交给同一个 GatewayCore; 不是主 Reactor 接连接后交给子 Reactor, 也不是每个循环各配一个业务线程。

共用业务线程是因为这些入口经常涉及同一设备、同一命令。业务处理顺序集中在一个线程, 链路窗口由串口 Reactor 独占, SQL 由独立读写线程负责, 资源准备和 MQTT 发布/替换由 control 线程串行管理。这样按状态归属和阻塞资源划分, 比仅按 MQTT、HTTP、串口入口各建业务池更容易解释一致性。

同一进程内的线程与消息交接

线程	独占职责	跨线程消息
串口 epoll	fd、缓冲、SR 状态	完整帧 / 命令完成
MQTT 网络循环	连接收发、MQTT 协议	原始 topic / payload
HTTP Asio 循环	连接、HTTP 解析和收发	请求 / 响应完成
GatewayCore	应用转换与路由	命令 / 发布 / SQL 任务
gateway-control	MQTT 发布/替换、资源准备	串口候选 / 有序切库任务
SQLite 读写执行器	各自连接与语句	查询结果 / 写入进度

回到第一部分

T013 · 进程、线程和 FreeRTOS 任务有什么区别？

T021 · Linux 调度与 FreeRTOS 调度有什么不同？

T022 · 中断上半部、下半部与任务化处理为什么重要？

T084 · FreeRTOS 的任务状态和调度链路如何理解？

T087 · 队列为什么复制数据，传指针又有什么代价？

实现依据与展开

三类常见模式的区别

单 Reactor 单线程让 I/O 和业务串行执行；单 Reactor 多线程把业务交给工作线程；多 Reactor 多线程让多个循环管理不同连接或通道，业务线程可共享。进程数量是另一个维度。Asio 对应用暴露异步完成接口，Linux 默认后端通常为 epoll，描述时须区分接口与实现层。

节点侧的对应关系

Cmd、Trans、Sample 优先级分别为 3、2、1，分别等待 StreamBuffer、发送队列或采样周期。它们共享 SRAM，节点 SR 状态由 Cmd 任务独占；此轮网关线程改造没有把节点改成 Reactor。

网关 GatewayApp.cpp:113 · 网关 GatewayApp.h:21 · 节点 app.c:13 · 节点 cmd_service.c:77 · 节点 tx_service.c:22 · 节点 report_task.c:55 · 网关 GatewayCore.h:20 · 网关 TaskExecutor.h:19

继续追问

为什么不先建多个业务线程提高并行度？

会增加设备状态、命令关联和顺序协调。先测业务排队与处理耗时；若确有瓶颈，可按设备分组，让同一设备各入口事件始终到同一业务线程。该扩展是后续方案，当前没有实现。

当前实现边界

多循环隔离了直接回调占用，但仍共享 CPU、内存与磁盘；线程更多不是性能一定更好的证据。当前选择同时考虑协议库已有事件循环及实现成本。

P005 项目问答

哪些资源由谁拥有，怎样避免多处同时操作？

面试时这样回答

串口 fd、输出缓冲和 CommandTracker/SR 状态由串口 Reactor 操作；MQTT 网络回调只交接消息，GatewayCore 做应用转换；HTTP 连接由 Asio 循环管理，业务线程通过完成函数返回响应。SQLite 写连接和读查询分别由专用执行线程串行使用；普通 MQTT 发布与客户端替换统一交 control 管理。

我会把“谁能访问”和“谁负责释放”分别说明。Channel 借用串口 fd，真正关闭 fd 的是串口所有者；HTTP 响应须活到异步写结束，连接关闭后迟到业务结果不能写到新连接。跨线程消息持有自身数据，避免把局部帧、topic 或 payload 的引用留给后续消费者。

回到第一部分

T001 · RAII 为什么比到处手动 close 更可靠？

T002 · unique_ptr、shared_ptr、weak_ptr 如何选择？

T005 · 引用、指针、const 和回调捕获该怎样解释？

T061 · 优雅退出为什么要先停止生产者，再排空消费者？

T087 · 队列为什么复制数据，传指针又有什么代价？

T112 · 零拷贝为什么首先是缓冲所有权问题？

实现依据与展开

所有权的关键证据

unique_ptr/RAII 管独占资源，shared_ptr/weak_ptr 或受控快照管理跨回调寿命；这不意味着被指对象可以任意并发调用。MQTT 发布与客户端替换也必须在一致生命周期协议下执行，停机先关闭入口再处理存量并等待消费者结束。

网关 GatewayApp.h:21 · 网关 EventLoop.cpp:58 · 网关 NodeLink.cpp:92 · 网关 TelemetryPipeline.cpp:45 · 节点 uart_link.c:41 · 节点 tx_service.c:22 · 节点 sensor_hub.c:19 · 网关 GatewayCore.h:20 · 网关 HttpServer.h:19

继续追问

有 shared_ptr 为什么还需要判断对象是否失效？

保活只保证内存还在。HTTP 连接已经关闭或串口会话已经变化时，旧回调即使访问有效内存，也不能继续推进新业务状态。

当前实现边界

当前节点 RX 仍通过复制交接数据，并未实现带块池与信用控制的接收零拷贝。

P006 项目问答

你怎样评价项目现在的可靠性？

面试时这样回答

我会把可靠性分层回答。串口控制链有校验、有限重试、会话隔离、按序执行和结果缓存，能处理规定范围内的丢帧与重复。传感器读取带有效性判断和部分错误恢复，看门狗用于发现关键任务长期没有签到。

但它还不是掉电后也能完整恢复的业务系统。节点的协议记录在 RAM，云端命令缺少持久业务 ID，遥测也没有统一的端到端确认。队列满、发布失败和数据库错误都有需要明确处理的路径，所以不能说“用了 QoS 1 和重传，数据就不会丢”。

回到第一部分

T039 · 传输确认、按序交付与业务完成为什么不是同一状态？

T043 · 去重缓存满了、过期了或结果 ACK 丢了会怎样？

T044 · 幂等、去重与恰好一次有什么区别？

T051 · 断网、重连、消息重复时怎样保证业务可追踪？

T056 · ACID、WAL 和断电安全有什么关系？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

实现依据与展开

一个准确的保证范围

同一有效会话内，不复用 seq，旧序号不会因为结果缓存释放而重新执行；具体结果只在有限缓存责任期内可重放。进程重启、节点掉电与跨新会话重复业务属于另一层恢复问题。

网关 edge_src:62 · 网关 CommandTracker.cpp:78 · 网关 MqttClient.cpp:21 · 网关 Database.h:110 · 节点 uart_link.c:41 · 节点 tx_service.c:22 · 网关 TelemetryPipeline.cpp:45

继续追问

最值得先补的可靠性能力是什么？

先把失败可见性补齐：接收溢出、TX 入队失败、驱动重启失败和数据库写入结果都要能观察与上报；随后按实际需求决定是否增加持久命令/outbox，而不是一开始承诺全部消息永久保存。

当前实现边界

上述改进方向是设计建议，不能当作当前已实现功能或已测得成果。

网关并发与 Linux I/O

P007 项目问答

串口 Reactor 与业务线程的职责怎样严格区分？

面试时这样回答

串口 Reactor 管非阻塞读写、增量拆帧、CRC，以及会话、序号、窗口、确认和重传这些链路状态。GatewayCore 管命令解释、遥测单位转换、MQTT topic/payload、业务结果映射和 HTTP 路由；SQLite 查询与写入再提交专用执行线程。

链路处理留在 Reactor，是因为它紧贴部分写完成、收到 ACK 和超时扫描，需要单一状态所有者。收到遥测帧或完成事件后只交接消息，不在串口回调中格式化业务结果。业务提交命令后继续消费其他事件，不阻塞等节点响应。

回到第一部分

T016 · 阻塞、非阻塞、同步、异步如何区分？

T026 · select、poll、epoll 的区别，epoll 真的是 O(1) 吗？

T029 · eventfd 为什么配一个队列，计数是否等于命令条数？

T030 · timerfd 怎样报告超时，时钟和定时精度如何选择？

T031 · 为什么用 signalfd，为什么要在创建线程前屏蔽信号？

T035 · 事件循环如何同时保证就绪处理、公平性与退出？

实现依据与展开

两类状态与两个时间点

CommandTracker 独占的是链路请求状态，串口整帧被内核接受后才更新本次发送时间。GatewayCore 消费的是已拥有存储的应用事件；帧完成不等于业务处理完成，业务完成也不等于 MQTT 对端已确认。

事件源依据

epoll 等待串口读写、eventfd 跨线程提交、timerfd 单调时钟扫描和 signalfd 控制信号；MQTT 与 HTTP 的网络循环独立运行。GatewayCore 等待任务队列，不监听 I/O 就绪，所以不另算一个 I/O Reactor。

网关 GatewayApp.cpp:113 · 网关 EventLoop.cpp:58 · 网关 CommandTracker.cpp:78 · 网关 NodeLink.cpp:92 · 网关 GatewayCore.cpp:118

继续追问

Reactor 是否必须完全不解析任何协议？

不需要。消息定界、CRC、HTTP/MQTT 解析和轻量链路状态属于通信层；业务字段解释、传感器单位、路由与 SQL 属于应用层。边界按责任和耗时界定，不以函数里有没有 if 或状态机来区分。

当前实现边界

串口 ET 排空和局部资源操作仍需单独分析耗时；采用多循环不证明所有高负载尾延迟已经验证。

P008 项目问答

MQTT 到业务线程再到串口，为什么需要两次消息交接？

面试时这样回答

MQTT 网络回调先把拥有独立存储的原始消息交给 GatewayCore；业务线程被任务队列的条件变量唤醒，解释命令后才提交串口下行队列并写 eventfd。第一段隔离 MQTT 收发与业务解析，第二段保证串口和链路状态只由串口 Reactor 修改。

串口 Reactor 被唤醒后以实际队列与链路许可为准消费命令，不把 eventfd 计数当作命令数量。一次只处理有限批次，仍有可处理任务就重新安排；会话未就绪或 SR 窗口满则等链路状态推进，避免没有发送能力时忙循环。

回到第一部分

T017 · 互斥量、信号量、条件变量和自旋锁怎么选？

T018 · 条件变量为什么要配谓词，如何避免丢失唤醒？

T029 · eventfd 为什么配一个队列，计数是否等于命令条数？

T035 · 事件循环如何同时保证就绪处理、公平性与退出？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

实现依据与展开

四个不同限制

业务入口任务数、下行排队命令数、单轮消费预算和 SR 在途窗口是不同层次的限制。入队必须非阻塞地接受或拒绝，不能为了保留每条消息而让网络回调等待队列空位；容量数字和满时结果应以对应队列构造与提交接口为准。

网关 GatewayApp.cpp:113 · 网关 ThreadSafeQueue.h:94 · 网关 MqttClient.cpp:21 · 网关 GatewayCore.cpp:118 · 网关 TaskExecutor.h:19

继续追问

为什么不按 `eventfd` 读出的数值恰好处理同样多命令？

多个通知可合并，消费者也可能主动续发通知；计数和业务条目不是一一对应。队列保存消息，`eventfd` 仅让事件循环知道可能有工作，不能替代容器同步和接纳策略。

当前实现边界

有界队列和有限批次不自动建立全链路背压。已经被 MQTT 协议接收但业务队列拒绝的消息，仍须按应用失败契约解释，不能声称 QoS 自动保证设备执行。

P009 项目问答

串口为什么用 `ET`，读到 `EAGAIN` 有什么意义？

面试时这样回答

串口使用非阻塞 `fd`，收到可读事件后循环读取，把每次实际读到的字节送进增量解析器，直到 `EAGAIN` 才返回。`ET` 不保证对已经留在内核里的剩余数据再提醒一次，所以不能只读固定一帧就退出等待。

这里一次 `read` 与一条协议帧没有对应关系。解析器保留半帧状态，能处理一次读到半帧或多帧。`EINTR` 会重试，`EAGAIN` 表示当前没有更多进展，其他错误则记录并结束这次读取。

回到第一部分

T026 · `select`、`poll`、`epoll` 的区别，`epoll` 真的是 $O(1)$ 吗？

T027 · `ET` 和 `LT` 的区别，为什么 `ET` 要读到 `EAGAIN`？

T028 · 串口短写、`EINTR`、`EAGAIN` 分别怎样处理？

T033 · 串口 `termios` 配置为什么重要？

T035 · 事件循环如何同时保证就绪处理、公平性与退出？

实现依据与展开

当前实现依据

`NodeLink::drainAndParse` 使用 256 字节临时缓冲并持续 `read`；串口注册 `EPOLLIN` 与 `EPOLLET`，`termios` 配置承担原始二进制模式和线参数设置。

网关 `NodeLink.cpp:92` · 网关 `SerialPort.cpp:93` · 网关 `GatewayApp.cpp:113`

继续追问

持续输入时读到 **EAGAIN** 会不会影响公平？

可能。当前读取循环没有独立字节/时间预算；若进一步限额，需要把未耗尽的 **fd** 保存在本地待处理集合，不能丢下后直接等下一次 **ET** 通知。

当前实现边界

选择 **ET** 的实现约束已经满足基本非阻塞排空路径，但不能据此宣称所有高负载下的尾延迟均已验证。

P010 项目问答

短写和 **EAGAIN** 时，怎样保证帧不会交错或重复前缀？

面试时这样回答

网关发送队列保存完整编码帧和当前偏移。一次 **write** 只接受部分字节，就只推进相应偏移；遇到 **EAGAIN** 保留剩余部分，等可写事件继续。队首帧没写完之前不发送后面的帧，所以不会把两条帧的字节交错。

整帧被内核接受后才通知命令状态机启动这一轮等待确认的计时。尚未开始的旧请求可以通过身份检查取消；已经写出一部分的帧不能随意扔掉后缀，否则会破坏对端字节流。

回到第一部分

T028 · 串口短写、**EINTR**、**EAGAIN** 分别怎么处理？

T032 · 回调删除连接时，如何避免 **use-after-free**？

T001 · **RAII** 为什么比到处手动 **close** 更可靠？

T041 · 怎样设计 **RTO**、总截止时间与有限重试？

T112 · 零拷贝为什么首先是缓冲所有权问题？

实现依据与展开

代码与完成边界

NodeLink::flushOutput 推进 **offset**；队列空后取消 **EPOLLOUT**。完成回调携带会话和发送尝试身份，**CommandTracker** 再检查其有效性。这里“写完”指本地内核接纳，不等于 **UART** 引脚已发完，更不等于节点执行。

网关 **NodeLink.cpp:92** · 网关 **CommandTracker.cpp:78** · 网关 **GatewayApp.cpp:113**

继续追问

为什么只在有待发数据时监听 EPOLLOUT?

串口通常长期可写，空队列时一直监听可能产生无意义唤醒。需要续写时打开，排空后关闭，能减少主循环空转。

当前实现边界

真正发送失败会清理对应输出并推进失败处理；当前输出队列容量也应与上游接纳限制一起分析。

P011 项目问答

回调里关闭连接，怎样避免访问已经释放的对象？

面试时这样回答

`epoll_wait` 一次可能返回多个 `Channel` 指针。处理前面的回调时，如果删除了后面某个 `Channel`，事件数组里仍保留它的旧指针。因此注销后先标记失效，并暂存一份所有权，等整批事件处理完再释放。

派发时跳过失效对象；一个读回调也可能删除自己，所以再调用写回调前还要检查一次。这里同时处理了逻辑失效和内存寿命，单纯从 `map` 删除或从 `epoll` 注销都不够。

回到第一部分

T032 · 回调删除连接时，如何避免 `use-after-free`?

T002 · `unique_ptr`、`shared_ptr`、`weak_ptr` 如何选择?

T005 · 引用、指针、`const` 和回调捕获该怎样解释?

T024 · `fd`、`open file description`、`dup` 与 `close` 有什么区别?

实现依据与展开

实现依据

`EventLoop::removeChannel` 设置 `dead`，把 `shared_ptr` 放入 `dying_` 并移出活动表；本轮事件循环结束后清空 `dying_`。

网关 `EventLoop.cpp:58`

继续追问

这种办法能保护线程池里很晚才运行的回调吗？

不能自动保护。它只覆盖当前事件批次；更长寿命的异步任务需要独立的共享/弱引用、代数校验或取消完成协议。

当前实现边界

这个设计针对单主线程中的批次回收，不是可以任意跨线程删除 Channel 的通用证明。

P012 项目问答

退出与重载怎样通过 **signalfd** 和跨线程屏障协作？

面试时这样回答

程序在线程启动前屏蔽要接管的信号，再让串口 Reactor 通过 **signalfd** 读取。**SIGTERM/SIGINT** 请求退出循环，**SIGHUP** 只投递重载任务给 **gateway-control**；文件解析、数据库构造、MQTT 重建等工作不在信号回调执行。

停机先设置入口停止状态、**shutdown** 下行与候选队列并关闭、等待 HTTP 循环，把已排队或在途命令交给业务线程形成 **shutdown** 终态。随后 **GatewayCore** 先等屏障前的业务提交完成，再排空查询线程，最后排空业务完成事件；**control** 排空后销毁 MQTT 并等待网络线程退出，写库管道在后续析构中排空。

回到第一部分

T031 · 为什么用 **signalfd**，为什么要在创建线程前屏蔽信号？

T035 · 事件循环如何同时保证就绪处理、公平性与退出？

T061 · 优雅退出为什么要先停止生产者，再排空消费者？

T108 · **systemd** 部署、日志与自动重启怎样配合？

实现依据与展开

为什么查询需要独立屏障

如果 **core** 先关闭，尚未完成的 SQL 就无法把结果投回；如果 **query** 先关闭，尚未运行的 HTTP 业务又可能继续提交 SQL。屏障确认此前 HTTP 已完成任务提交，再关闭 **query**，最后关 **core**，保证生产者与消费者的依赖顺序。等待仅由退出所有者执行，正常 Reactor/业务任务不等待。

网关 **main.cpp:21** · 网关 **GatewayApp.cpp:113** · 网关 **GatewayApp.h:21** · 网关 **ThreadSafeQueue.h:94** · 网关 **MqttClient.cpp:21** · 网关 **GatewayCore.cpp:118** · 网关 **TaskExecutor.h:19**

继续追问

MQTT 为什么最后才 **join**，是否还会投递到已经关闭的业务对象？

stopping 先让网络入口返回，已进入回调的尝试由 **Core accepting** 和邮箱关闭协议拒绝；**Core** 对象一直保留到 MQTT 停止之后。HTTP 迟到回复则由失效的 **ReplyBridge** 丢弃，不能向已销毁的 **io_context post**。

当前实现边界

排空邮箱只证明任务已处理或交给下一层，不保证 MQTT PUBACK、远端业务执行或数据库提交全都成功。`control` 中同步连接等待也会影响停机耗时，当前没有无限负载下的固定退出上界。

P013 项目问答

C++ 的 RAII、移动语义和共享快照在项目中分别起什么作用？

面试时这样回答

RAII 负责让串口、线程和连接等资源跟着所有者生命周期释放；移动语义把完整帧或连接交给后续消费者，避免不必要复制；共享快照则用于让正在运行的读者继续持有旧版本。三者分别解决资源释放、交接成本和并发读寿命。

例如串口发送队列持有完整 `vector`，不借用临时组帧数组；写库线程持有自己的数据库连接；配置更新时发布新快照，已经取得旧快照的操作可以继续完成。只靠一个裸指针或把变量声明为 `volatile`，做不到这些保证。

回到第一部分

T001 · RAII 为什么比到处手动 `close` 更可靠？

T002 · `unique_ptr`、`shared_ptr`、`weak_ptr` 如何选择？

T003 · `std::move` 做了什么，移动后对象还能用吗？

T004 · `vector` 扩容、迭代器失效和字符串缓冲有什么坑？

T005 · 引用、指针、`const` 和回调捕获该怎样解释？

T059 · 配置快照、资源切换与事务原子性是什么关系？

实现依据与展开

版本依据

工程使用 C++17。配置/读连接的 `shared_ptr` 使用对应原子加载与存储接口，而不是 C++20 的 `atomic<shared_ptr<T>>` 写法；共享计数也不保护 `Database` 对象被任意多线程并发调用。

网关 `GatewayApp.h:21` · 网关 `NodeLink.cpp:92` · 网关 `TelemetryPipeline.cpp:45` · 网关 `Config.cpp:134`

继续追问

为什么不所有对象都用 `shared_ptr`？

唯一所有者用独占对象更清楚，也避免引用环和额外共享状态。只有确实需要多个使用者跨版本共同保活时才引入共享所有权。

当前实现边界

RAII 能帮助正常退出和异常展开，不能保证强制杀进程或断电时执行清理。

P014 项目问答

为什么换成 **Boost.Beast**，HTTP 与业务线程怎样配合？

面试时这样回答

HTTP 协议解析和异步收发改用 **Boost.Beast/Asio**。独立线程运行 `io_context`，连接用 `async_read` 跨读取拼出完整请求，再把拥有自身存储的请求投递 `GatewayCore`。路由与参数校验在业务线程执行，历史数据查询提交独立读执行线程。

查询完成后生成响应，通过完成函数投递回连接所属 `io_context`，最终 `async_write` 发送。连接对象和响应对象保持到相应回调结束，慢查询不占住 HTTP 事件循环，业务线程也不通过 `future.get` 等待查询。第三方库减少自行维护 HTTP 定界、错误解析与短写的负担。

网络循环与业务、数据库执行分离

Beast 完成 HTTP
读取

GatewayCore 校验
路由

读执行器执行 SQL

GatewayCore 生成
响应

post 回 HTTP /
`async_write`

回到第一部分

T034 · HTTP 如何处理半包、连续请求和慢客户端？

T046 · TCP 为什么还需要应用层分帧？

T054 · SQLite 的连接、读写并发与线程模式怎么理解？

T057 · 索引、预编译语句和查询上限为什么重要？

实现依据与展开

责任分界

Beast 负责 HTTP 消息层，Asio 负责异步调度与 `socket`；应用仍负责路由和任务准入。请求头上限 16 KiB、正文上限 64 KiB、同时连接上限 256，每连接一次只等待一个业务结果；读写与业务等待各设期限。

ReplyBridge 用弱引用和短锁控制停机后的投递，`request_id` 拦截重复或迟到回复。

网关 `HttpServer.cpp:99` · 网关 `GatewayApp.cpp:113` · 网关 `GatewayCore.cpp:118` · 网关 `HttpServer.h:19`

继续追问

每个 HTTP 请求是否都建立一个新线程？

不是。多个连接共享 HTTP I/O 线程；GatewayCore 共享业务线程，SQL 共享有界读执行器。每个连接保存自己的解析、等待和响应状态，当前请求完成后才推进后续响应顺序。

当前实现边界

引入第三方 HTTP 库不自动获得 HTTPS、身份认证或无限连接能力。前端页面仍属于既有外部贡献，不归为本次 HTTP 后端重构成果。

帧协议、命令可靠性与 MQTT

P015 项目问答

串口帧怎样设计，为什么还要增量解析和 CRC？

面试时这样回答

帧包含同步头、长度、类型、业务负载和 CRC。同步头帮助寻找候选起点，长度限定正文边界，CRC 检查内容是否损坏。接收端按字节推进解析状态，不依赖一次 read 或一次 DMA 回调恰好得到一条帧。

当前同步头是 AA55，LEN 包含 TYPE，允许 1 到 64，因此完整帧为 6 到 69 字节。CRC 使用 CRC16/MODBUS 参数，覆盖 LEN、TYPE 和 payload，结果低字节先传；业务多字节整数则按协议大端编码。CRC 端序与业务端序是分别约定的。

回到第一部分

T006 · 结构体为什么不能直接当作串口协议发送？

T036 · 如何设计一条能抗分包和噪声的串口帧？

T037 · CRC16 与累加和有什么区别，能保证不出错吗？

实现依据与展开

避免混淆的两个校验

DHT11 原始数据自身带累加校验，串口帧外层另有 CRC16。使用这个 CRC 变体不表示整个自定义协议就是 Modbus，也不意味着具备消息认证。

网关 edge_proto.h:110 · 网关 edge_frame.c:101 · 网关 edge_crc16.c:10 · 网关 NodeLink.cpp:92

继续追问

负载里出现 AA55 会不会误切帧？

正常收正文时按当前长度继续，不应看到同步头就重启。只有处于寻头/恢复阶段才按同步规则找下一候选位置，错误长度和 CRC 失败还需有界重同步。

当前实现边界

CRC 通过说明满足校验，不等于字段和业务参数都合法；类型、长度、返回码和范围仍须检查。

为什么用 SR，窗口为什么设成 8？

面试时这样回答

SR 允许多条命令在途，并对缺失项单独重传，比每发一条都等完结果更能利用链路。节点可以先保存乱序到达的后续命令，但只按序号顺序执行业务，避免控制顺序被乱序传输打乱。

窗口 8 是当前资源和状态复杂度下的配置选择，不是由 8 位序号强制算出来的最佳值。节点有 16 个槽共同保存待执行命令和已完成结果，结果还要等确认或到期释放，所以窗口不能只按序号位数放大。

回到第一部分

T038 · 停止等待、GBN 与 SR 怎样工作，窗口上限如何推导？

T039 · 传输确认、按序交付与业务完成为什么不是同一状态？

T043 · 去重缓存满了、过期了或结果 ACK 丢了会怎样？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

实现依据与展开

两个限制共同生效

网关既限制发送序号不能超过接收窗口范围，也限制未完成业务数量。经典循环 SR 的半序号空间规则用于理解歧义；本协议同一 session 内不复用 seq，用完后排空并换会话。

网关 edge_proto.h:110 · 网关 CommandTracker.cpp:78 · 网关 edge_src.c:62

继续追问

窗口加倍，吞吐会加倍吗？

不一定。UART 带宽、节点串行执行和传感器转换都可能成为瓶颈，更大窗口可能只增加积压和结果缓存压力，需要按实际负载测量。

当前实现边界

当前配置不构成全部最坏突发下 RX 缓冲不溢出的证明。

接收确认、执行结果和结果确认为什么要分开？

面试时这样回答

接收确认告诉网关“这条请求已保存，可以停止接收阶段的重传”；执行结果告诉网关“业务已经结束，结果是什么”；结果确认告诉节点“网关已经拿到结果，可以释放缓存”。三个消息对应三种责任，不能合成一个含糊的 ACK。

节点执行完先保存结果、推进执行序号，再尝试发送结果。这样 TX 队列暂时满了，也不会因为同一请求重来就再执行一次。结果确认影响缓存回收，不决定下一条业务是否可以执行。

四个里程碑分别证明什么

事件	方向	含义
本地整帧提交	网关 → 本地内核	启动本轮接收等待；尚未证明对端收到
SR_RECEIVED	节点 → 网关	节点已接纳请求；尚未证明执行成功
业务结果	节点 → 网关	链路接收终态；投递 Core 生成应答
SR_RESULT_ACK	网关 → 节点	网关链路已接收；不证明 MQTT 已交付

回到第一部分

T039 · 传输确认、按序交付与业务完成为什么不是同一状态？

T043 · 去重缓存满了、过期了或结果 ACK 丢了会怎样？

T044 · 幂等、去重与恰好一次有什么区别？

T049 · MQTT QoS 0、1、2 到底保证什么？

实现依据与展开

非理想到达顺序

网关允许结果先于接收确认到达，合法结果可同时证明接收已发生；重复结果补发确认但不重复完成上层请求。当前串口 Reactor 先接收链路结果、交付 Core 完成事件并推进结果确认，不等待 Core 格式化或 MQTT 发布，因此 SR_RESULT_ACK 不能证明云端已收到应答。

网关 CommandTracker.cpp:78 · 网关 edge_sr.c:62 · 节点 cmd_service.c:77 · 网关 GatewayApp.cpp:113 · 网关 GatewayCore.cpp:118

继续追问

节点发出接收确认后立刻掉电，会怎样？

请求只在 **RAM** 中保存，掉电后记录丢失。这个确认不是持久接纳保证，不能据此承诺跨掉电完成。

当前实现边界

缓存寿命、节点重启与上层业务重复的范围必须与链路确认分开说明。

P018 项目问答

会话号怎样防止旧命令污染新连接？

面试时这样回答

请求身份由 64 位 **session** 和 8 位 **seq** 共同组成。**session** 区分不同一轮链路，**seq** 区分本轮请求；双方先通过 **OPEN** 握手建立会话，再接纳该会话的命令。旧会话的确认、结果和写完成回调不能直接修改新会话状态。

节点在本次启动期间保留会话高水位，更高 **session** 才能开启新一轮并清理旧状态。同一会话内 **seq** 不复用，序号用完且在途清空后再建立新会话，减少迟到短序号被误认成新请求的机会。

回到第一部分

T040 · 会话号与序号为什么缺一不可？

T007 · 整数溢出、符号转换和时间回绕有哪些隐蔽问题？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

握手拒绝如何处理

OPEN_ACK 包含节点认知的高水位；网关能据有效拒绝推进会话值，但仍受握手次数与总期限限制，不能被连续拒绝拖成无限重试。

网关 `edge_proto.h:110` · 网关 `CommandTracker.cpp:78` · 网关 `edge_src.c:62`

继续追问

会话号能作为永久业务 **ID** 吗？

不能。重连后的相同业务可能得到新会话/序号，节点掉电也会失去高水位。跨重启去重需要上层稳定 **ID** 与持久记录。

当前实现边界

64 位宽度降低身份碰撞风险，不提供认证；未认证链路仍有伪造消息的边界。

P019 项目问答

500 ms、4 s、10 s 三个时间分别控制什么？

面试时这样回答

500 ms 是一轮命令整帧写入内核后等待接收确认的重试间隔，最多额外重试 3 次。4 s 限制未取得接收确认的请求总寿命，避免在本地排队或持续重试中无限占资源；10 s 是从请求被状态机接纳排队起等待业务结果的总期限。

收到接收确认后不再按“命令没到”重复发送，但仍保留业务对象等待结果。网关每 200 ms 扫描一次，再加主循环延迟，所以实际报告超时会晚于名义阈值；这些值是判定条件，不是精确的完成时间承诺。

回到第一部分

T030 · timerfd 怎样报告超时，时钟和定时精度如何选择？

T041 · 怎样设计 RTO、总截止时间与有限重试？

T100 · 忙等、任务延时、硬件计时怎样选择？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

为什么从写完开始单轮计时

发送可能遇到短写或 EAGAIN，如果入队就开始每轮重试，尚未发出的帧也可能被重复排队。总寿命约束排队，写完成点约束这一轮等待，两者分别解决问题。

网关 CommandTracker.cpp:78 · 网关 edge_proto.h:110 · 网关 GatewayApp.cpp:113 · 网关 NodeLink.cpp:92

继续追问

把 500 ms 改为 50 ms 能更快完成正常命令吗？

正常响应不会因此提前，反而可能在节点合法处理延迟内制造重复。先测线路、传感器、队列和调度延迟，再选择覆盖正常波动的阈值。

当前实现边界

请求从 MQTT 收到、业务任务排队到被 CommandTracker 接纳之间的等待，不应误说成已经受同一组期限完整约束。

一条命令重试耗尽，为何中止整轮会话？

面试时这样回答

节点要求按序执行。如果缺失的是 5，后面的 6、7 即使已经收到，也不能跳过 5 执行。网关只删除 5 的本地记录，节点仍会等这个缺口，所以需要双方重建一致的顺序起点。

当前做法是结束旧会话，让相关请求得到失败或会话中止结果，再用更高 **session** 建立新会话。旧命令不会自动复制到新会话重发，因为其中一些可能已经执行，只是结果没回来。

回到第一部分

T042 · 有序协议缺口无法补齐时，怎样恢复会话？

T040 · 会话号与序号为什么缺一不可？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

残留发送的处理

发送前检查会话与尝试身份；未开始的旧项可取消，部分已写出的帧需要遵守流完整性处理。晚到的旧确认和回调同样经过身份校验。

网关 `CommandTracker.cpp:78` · 网关 `edge_src:62` · 网关 `GatewayApp.cpp:113` · 网关 `NodeLink.cpp:92`

继续追问

为什么不自动补发所有旧命令，用户体验更好？

对增量动作可能重复产生物理效果。可以由业务层查询状态、设置绝对目标或使用持久业务 ID，但不能让传输层自行猜测哪些动作没发生。

当前实现边界

会话恢复恢复的是接纳新命令的能力，不是撤销旧物理动作。

结果缓存满了、过期了、结果确认丢了，分别会怎样？

面试时这样回答

节点的 16 个槽同时存待执行请求和已完成结果。没有空槽时不确认未保存的新命令，让发送方继续按协议重试或终止；执行后结果从完成时刻起保留 5 s，收到结果确认可以提前释放，重复请求不会无限续期。

结果确认丢失时，节点在规定次数内重发结果，网关对重复合法结果再次确认。缓存过期后，旧序号仍在执行基序号左侧，所以不会重新执行，但可能已经无法返回原来的具体结果。

回到第一部分

T043 · 去重缓存满了、过期了或结果 ACK 丢了会怎样？

T039 · 传输确认、按序交付与业务完成为什么不是同一状态？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

实现依据与展开

不重复与可追溯要分开

执行基序号保留“已经越过这个请求”的信息，结果槽保留“这次具体返回了什么”。二者寿命不同，不能把有限结果缓存称为永久结果查询服务。

网关 `edge_src.c:62` · 网关 `edge_proto.h:110` · 网关 `CommandTracker.cpp:78`

继续追问

相同 `session/seq` 但参数不同会怎样？

只要相应记录仍存在，就比对命令类型、参数长度和内容，不一致不按原请求接纳；缓存释放后只能依据保留的序号历史拒绝重执行，无法凭空重建旧参数。

当前实现边界

这些记录在 RAM，掉电后不能继续提供同样保证。

MQTT 使用什么 QoS，发布与重建由谁管理？

面试时这样回答

命令订阅与命令结果使用 QoS 1，遥测使用 QoS 0。libmosquitto 网络线程处理 MQTT 收发和回调，消息回调只提交原始数据给 GatewayCore，业务解析和结果格式化都在业务线程。CONNACK 成功后重新订阅，连接 API 返回、收到 CONNACK 与订阅成功是不同阶段。

GatewayCore 生成的发布任务交给单一 gateway-control 线程；MQTT 客户端替换也在该线程执行，避免业务线程借用正在销毁的客户端。网络入口过载时直接通过回调所属客户端发送 gateway_busy，这是固定接纳失败处理；实际网络收发仍由库推进。

回到第一部分

T049 · MQTT QoS 0、1、2 到底保证什么？

T050 · Clean Session、Retain、Keep Alive 和遗嘱分别是什么？

T051 · 断网、重连、消息重复时怎样保证业务可追踪？

实现依据与展开

生命周期与可靠性分别看

普通 publish 与资源替换按 control 邮箱顺序执行。MqttClient 构造调用同步 mosquitto_connect，首次创建和热重载都在 control 执行，启动主线程通过 future 等待初始化，运行中的 Reactor 不等待；析构 disconnect/loop_stop 等待网络线程。publish 返回仅说明库是否接受，当前没有把 PUBACK 关联成持久业务完成，也没有遥测 outbox。

网关 MqttClient.cpp:21 · 网关 GatewayApp.cpp:113 · 网关 GatewayCore.cpp:118 · 网关 TaskExecutor.h:19

继续追问

拆线程后网络重建就不会影响任何消息了吗？

不会直接占用串口或 HTTP 循环，但 control 同时承担发布提交，慢连接或旧客户端退出会延迟其后发布并可能压满队列。应观察队列与错误，不把资源所有权正确误说成永不丢消息。

当前实现边界

QoS 1 不能保证节点业务恰好执行一次；客户端队列、接纳额度、重启和断网恢复都需要单独约定。此轮没有新增可靠持久补传能力。

云端重复发送同一命令，会不会重复执行？

面试时这样回答

如果是同一条串口请求因 ACK 丢失而重传，`session/seq` 保持不变，节点按协议识别重复，不再执行业务。但如果云端重新发来同样文本，网关可能把它当作新的业务提交，分配新的链路身份，节点就会把它视为新命令。

所以链路去重和业务去重必须分开。要处理云端重发，需要让云端携带稳定业务 ID，网关或最终执行者持久记录它与参数及结果的关系；同 ID 不同参数应拒绝，而不是按新命令执行。

回到第一部分

T044 · 幂等、去重与恰好一次有什么区别？

T040 · 会话号与序号为什么缺一不可？

T049 · MQTT QoS 0、1、2 到底保证什么？

T051 · 断网、重连、消息重复时怎样保证业务可追踪？

实现依据与展开

当前接口依据

`CommandTranslator` 解析出操作和参数，现有流程没有持久 `inbox/outbox`，也没有跨会话稳定业务 ID 的全链路约定。

网关 `CommandTranslator.cpp:57` · 网关 `GatewayApp.cpp:113` · 网关 `CommandTracker.cpp:78` · 网关 `edge_sr.c:62`

继续追问

设置采样周期是幂等操作，是否就不用考虑重复？

设置同一目标值的主要效果相同，但仍要考虑返回结果、日志、定时相位及未来附带副作用。查询又可能返回不同时间的测量值，不能把所有命令统一处理。

当前实现边界

持久业务去重是后续设计方向，当前代码不能证明跨掉电恰好一次。

项目各处队列满时怎么办，完成事件怎样预留空间？

面试时这样回答

GatewayCore 普通邮箱上限 1024，命令准入最多 512，查询最多 32；完成事件另可使用 552 个预留位置。命令占据准入额度直到 control 执行拒绝或终态消息的 publish 调用，再释放额度，避免已接收命令结果被新的遥测挤掉。串口下行队列容量 512，一轮最多消费 32，SR 在途仍最多 8。

普通邮箱满时遥测拒绝并记日志，HTTP 返回 503；MQTT 未接纳命令回 gateway_busy。查询过多也返回 503。control 普通队列和写库队列各为 1024；control 另预留 528 个位置，其中 512 对应未发布终态额度，16 是控制余量。遥测不能挤占这些位置，但库拒绝或网络中断仍不保证结果已上传。

回到第一部分

T035 · 事件循环如何同时保证就绪处理、公平性与退出？

T043 · 去重缓存满了、过期了或结果 ACK 丢了会怎样？

T087 · 队列为什复制数据，传指针又有什么代价？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

T112 · 零拷贝为什么首先是缓冲所有权问题？

实现依据与展开

容量证明的前提

预留为 $512+32+8$ ，分别覆盖已准入命令的单次终态、查询完成及生命周期控制余量。Core 的 pending 请求 ID 去重与 Reactor 每请求只交付一次终态是该估算的前提。TaskExecutor 关闭后拒绝新任务、按 FIFO 排空已有项，生产者不等待空位；当前并非无锁容器。

节点与网关单位不同

节点 TX 为 8 个 TxFrame，RX DMA 和 StreamBuffer 各 64 字节，SR 请求/结果槽为 16。节点 TX 满可能丢新帧，StreamBuffer 短写仍须检查调用点；条目数不等于字节容量，网关队列限制不能自动保护节点所有缓冲。

网关 GatewayApp.cpp:113 · 网关 GatewayCore.h:20 · 网关 TaskExecutor.h:19 · 网关 TelemetryPipeline.h:30 · 节点 uart_link.c:41 · 节点 tx_service.c:22 · 网关 edge_proto.h:110

继续追问

预留队列是否意味着永远不丢命令结果？

只在准入上限、每项完成次数与停机顺序都成立时保护这一层。命令额度覆盖到 **control** 执行发布调用；其后还有 **libmosquitto** 和网络，库拒绝或掉线仍可能丢发布。需要持久业务 **ID/outbox** 才能进一步建立可恢复交付。

当前实现边界

当前没有全链路统一信用和持久背压，不能把几个有界队列等同于系统已不会过载。容量是代码配置和设计推导，尚不是板上突发流量测量结论。

数据存储、热加载与运维

P025 项目问答

为什么 SQLite 分成专用写线程和有界读执行器？

面试时这样回答

写库可能等待事务和磁盘，查询也可能很慢，所以两者都不能直接放在 I/O 回调或 GatewayCore 业务任务中。写连接由 TelemetryPipeline 写线程串行使用；HTTP 业务提交查询任务，由独立读执行器使用只读连接并异步返回值对象。

数据库尝试启用 WAL，允许条件成立时读写并行，但仍受单写者、锁和检查点限制。一个读线程能避免同一读连接的语句状态被多个线程同时使用；有界任务队列则使持续慢查询时可以明确拒绝新任务，而不是无限堆积。

回到第一部分

T017 · 互斥量、信号量、条件变量和自旋锁怎么选？

T054 · SQLite 的连接、读写并发与线程模式怎么理解？

T056 · ACID、WAL 和断电安全有什么关系？

实现依据与展开

应用层契约

Database 内部不加锁，同一实例须串行使用。shared_ptr 快照只延长已取得连接的寿命，不代替连接内同步；查询排队时保留哪个版本，由提交时快照与切库交接顺序决定。

网关 TelemetryPipeline.cpp:45 · 网关 Database.h:110 · 网关 GatewayApp.cpp:113 · 网关 GatewayCore.cpp:118

继续追问

HTTP 查不到刚采的记录，是不是线程不安全？

不一定。可能业务事件尚未处理、写任务仍排队或事务失败；先区分每个完成阶段，再定位读写连接切换，不能把异步可见延迟直接当成锁错误。

当前实现边界

WAL 设置失败仍记告警而非整体构造失败；独立读线程也不证明慢查询没有资源影响，需要观察查询耗时、队列拒绝和停止时间。

P026 项目问答

批量写库怎样兼顾吞吐与低负载延迟？

面试时这样回答

写线程先阻塞等第一项数据，取得后立即继续取当前已经排队的任务，合并成一批写入。暂时没有更多数据就提交，不额外等一个固定攒批窗口，所以低负载的一条数据不用为了凑数量再等几十毫秒。

有积压时自然合批，减少事务次数；累计达到行数阈值或遇到切库任务，就先提交当前批。切库是顺序边界，边界前的数据留在旧库，不能为了多合几行把它跨过去。

回到第一部分

T055 · 批量写入为什么能提速，怎样控制额外延迟？

T017 · 互斥量、信号量、条件变量和自旋锁怎么选？

T018 · 条件变量为什么要配谓词，如何避免丢失唤醒？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

实现依据与展开

配置与单位

队列容量按 Job 计，阈值为 256 行；一个 Job 可包含多个读数，所以加入最后一个 Job 后行数可能超过阈值，不能把它说成严格每批最多 256 行。

网关 TelemetryPipeline.cpp:45 · 网关 TelemetryPipeline.h:30 · 网关 Database.h:110 · 网关 ThreadSafeQueue.h:94

继续追问

为什么队列不用一直轮询？

没数据时条件变量让写线程休眠，避免空转；有数据后再机会式排空，兼顾唤醒次数与延迟。

当前实现边界

提高合批效率不修复底层数据库错误处理；写入结果仍要按 Database 的真实契约解释。

现在数据库能保证整批成功和断电不丢吗？

面试时这样回答

不能这样保证。SQLite 本身提供事务能力，但当前封装在 BEGIN 失败后仍继续逐条尝试，单条 INSERT 失败只记日志并继续，COMMIT 失败再尝试回滚；接口也没有返回具体成功数量。因此上层无法据一次函数返回判断这一批是否全部可靠写入。

另外，WAL 配合 NORMAL 主要平衡性能与安全，整机掉电时最近提交仍可能回退。进程崩溃和整机断电是不同故障，不能因为数据库通常能恢复，就宣称刚收到的所有遥测都已持久保存。

回到第一部分

T056 · ACID、WAL 和断电安全有什么关系？

T051 · 断网、重连、消息重复时怎样保证业务可追踪？

T115 · 错误码、异常安全与恢复重试怎样形成一致契约？

实现依据与展开

如果要加强该契约

应先决定批次全有全无还是允许部分成功，检查 BEGIN/bind/step/reset/COMMIT，并返回可用于重试的结果；若还要可靠发布 MQTT，则需要另设计事务内 outbox 和重复处理。

网关 Database.h:110 · 网关 TelemetryPipeline.cpp:45

继续追问

为什么不直接在错误后重放整批？

现有接口没有可靠告诉上层哪些行已提交，盲目重放可能重复。需要稳定记录身份或明确回滚结果，再谈安全重试。

当前实现边界

上述改造尚未实现，本问应如实解释当前能力，而不是把数据库的一般性质当成封装已具备的保证。

HTTP 查询为什么按设备和时间建联合索引？

面试时这样回答

常见查询是某个指标最近若干条数据，所以索引按 `device_id`、`ts` 组织。数据库可先定位指标，再按时间方向读取有限记录，减少全表扫描和排序；结果数量也有接口层约束，避免一次查询把大量历史数据搬进内存。

参数通过预编译语句绑定，返回给调用者的是复制好的值对象，不借用 SQLite 当前行的临时缓冲。预编译解决参数解释和部分重复解析成本，索引解决数据访问路径，两者作用不同。

回到第一部分

T057 · 索引、预编译语句和查询上限为什么重要？

T034 · HTTP 如何处理半包、连续请求和慢客户端？

实现依据与展开

源码依据

查询为 `WHERE device_id=? ORDER BY ts DESC LIMIT ?`；索引创建失败仅记告警，因此运行期还应检查真实 schema 和 `EXPLAIN QUERY PLAN`，不能只看建索引代码存在。

网关 Database.h:110 · 网关 GatewayApp.cpp:113 · 网关 GatewayCore.cpp:118

继续追问

为什么不用把所有字段都建索引？

每个索引都增加写入维护与空间成本。应针对实际查询证明收益，遥测持续写入时尤其要衡量写放大。

当前实现边界

当前查询接口对错误可能返回空或截断结果，不能把空集合一律解释为“没有数据”。

配置热加载怎样离开 Reactor，还剩哪些非原子边界？

面试时这样回答

SIGHUP 由串口 Reactor 读取后仅调用 `requestReload`。gateway-control 顺序执行配置读取、数据库与 MQTT 构造、串口候选准备；同一重载未完成时合并新请求。准备成功的串口通过队列和 `eventfd` 交回 Reactor，只有 `termios` 应用、`fd` 注册、链路状态失效和资源交接在所属线程执行。

这仍不是所有资源的事务切换。`ConfigManager` 先发布新快照，再分别处理资源；数据库等写线程处理 `SwapDb` 才生效，串口等 Reactor 消费替换任务才生效，MQTT 还要区分连接 API 返回和 `CONNACK`。某项失败可以保留旧资源，HTTP 监听端口目前也不会动态重绑。

回到第一部分

T059 · 配置快照、资源切换与事务原子性是什么关系？

T002 · `unique_ptr`、`shared_ptr`、`weak_ptr` 如何选择？

T019 · `volatile`、原子操作和内存屏障有什么区别？

T115 · 错误码、异常安全与恢复重试怎样形成一致契约？

实现依据与展开

同步操作移到了哪里

Database 构造、最多等待 200 ms 的切库入队、同步 `mosquitto_connect`、旧网络循环停止和串口 `open` 在 `control` 执行；共享 tty 的 `termios` 配置由 Reactor 在交接时应用。`control` 还负责发布提交，因此这些操作可能拖住它自己的后续任务；移出 Reactor 不等于消除所有等待。

网关 GatewayApp.cpp:113 · 网关 Config.cpp:134 · 网关 TaskExecutor.h:19

继续追问

reload done 是否表示所有外部连接已经应用成功？

不能。它汇总资源准备或任务提交结果，串口/数据库可能尚待交付，MQTT 尚待协议确认。完整配置代次、每模块应用状态与跨资源回滚仍是后续设计方向。

当前实现边界

此轮没有实现全局原子热加载，也没有测量重载期间的最大消息延迟。`ConfigManager` 先发布快照；某项资源应用失败后，原样再次 SIGHUP 可能因没有配置差异而不重试该项。只能根据执行线程证明这些构造与 `join` 不再直接运行于串口事件回调。

切换数据库时，队列里已经存在的数据写到哪里？

面试时这样回答

gateway-control 先构造候选读写连接，再把 SwapDb 放入遥测写队列。写线程处理到切库指令时先提交旧批次，再换写连接，指令前后数据分别按队列顺序进入旧库和新库；入队成功只是接纳切换任务。

读连接在写线程完成切换后的回调中原子发布。GatewayCore 提交查询时取得读连接快照，query 线程用该快照执行；已经排队的查询可继续读旧库，后续取得新快照的查询才读新库，不会因为全局指针替换而失去连接。

回到第一部分

T002 · unique_ptr、shared_ptr、weak_ptr 如何选择？

T055 · 批量写入为什么能提速，怎样控制额外延迟？

T060 · 如何有序切换后台资源而不破坏读写一致性？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

三个时间点与线程

候选 Database 构造在 control，SwapDb 入队也在 control，真正切换与读快照发布在 writer。最多 200 ms 的入队等待不限制此前构造耗时；共享指针延长寿命，不代表同一连接支持并发查询。

网关 GatewayApp.cpp:113 · 网关 TelemetryPipeline.cpp:45 · 网关 Database.h:110 · 网关 GatewayCore.cpp:118

继续追问

返回成功时，能立即断言所有查询都在新库吗？

不能。写切换可能还在排队，既有查询也可能持有旧快照。要看应用完成事件，并明确是否允许旧请求继续完成；当前允许它们使用提交时取得的版本。

当前实现边界

切换点按 TelemetryPipeline 的实际入队顺序定义，不是串口收帧与 SIGHUP 的全局时间切点；此前收到但仍在 Core 等待的遥测可能在 SwapDb 之后入队。队列顺序不补足 insertBatch 缺少逐批成功反馈的问题，HTTP 网络收请求时也未固定读快照。

串口热重开怎样准备候选并由 Reactor 安全接管？

面试时这样回答

gateway-control 在后台构造新的 NodeLink，仅完成串口 open 并延迟 termios 设置，再把候选放入有界替换队列并通知 eventfd。准备失败或替换队列满时不交付候选，当前 Reactor 继续持有旧链路；不会由 control 直接修改旧链路的窗口或 epoll 注册。

Reactor 收到候选后注销旧 Channel，调用 activate 应用 termios；失败恢复旧注册，成功则结束旧会话并暂存协议动作，接管候选并注册实际 fd 后才推进新握手。旧 NodeLink 析构丢弃输出，旧请求终态由 start 生成的动作交给 Core。新对象有自己的解析与发送状态，旧线路的半帧和输出不能直接拼到新线路上。

回到第一部分

T024 · fd、open file description、dup 与 close 有什么区别？

T032 · 回调删除连接时，如何避免 use-after-free？

T059 · 配置快照、资源切换与事务原子性是什么关系？

T060 · 如何有序切换后台资源而不破坏读写一致性？

实现依据与展开

所有权交接依据

候选 NodeLink 在 control 只完成 open，交付后由 Reactor 配置线路、设置 frame handler 和注册 Channel；候选打开成功不等于已经接管。Channel 借用 fd，实际关闭权仍属于链路内部串口对象，事件批次保活避免回调访问已删除 Channel。

网关 GatewayApp.cpp:113 · 网关 NodeLink.cpp:92 · 网关 SerialPort.cpp:93

继续追问

为什么不能拿旧 fd 整数继续用？

fd 是可复用编号，旧编号可能已被关闭或分配给另一资源；必须同时维护注册、回调寿命、协议会话与真正资源所有者，而不能仅更新一个整数。

当前实现边界

候选准备不会提前修改同一 tty 的共享 termios；属性修改与切换在 Reactor 串行执行。热重开仍是链路中断边界，此轮没有上板证明重开期间绝不丢输入，未写完帧可能被放弃并通过新会话恢复。

异步日志具体减少了什么开销？满了和退出时怎么办？

面试时这样回答

业务线程把日志追加到内存缓冲，后台线程批量写出。关键是只在锁内交换缓冲所有权，可能阻塞的 `write` 放在锁外执行，因此磁盘或标准错误输出较慢时，不会直接把那段写入时间都变成业务线程持锁时间。

缓冲和队列都有上限。后台跟不上时，当前实现裁剪待写队列尾部的缓冲块，并记录丢弃数量；析构会通知后台退出、等待线程，并尝试排空剩余内容。这个选择控制了内存增长，但日志不是无损审计存储。

回到第一部分

T018 · 条件变量为什么要配谓词，如何避免丢失唤醒？

T035 · 事件循环如何同时保证就绪处理、公平性与退出？

T058 · 为什么异步日志要在锁外写盘，队列满怎么办？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

实现依据与展开

具体机制

常规数据块为 64 KiB，前台使用当前块与备用块，后台交换得到本地批次后独占写出；低负载由刷新等待超时推动输出。超过单行上限的日志还会截断。

网关 `AsyncLogger.cpp:89`

继续追问

析构排空是否意味着已经落到物理介质？

不是。`write` 成功通常只证明内核接纳；实例默认写 `stderr`，后续持久性还取决于 `journald` 等收集器。当前也没有为每行提供写出确认。

当前实现边界

分配、锁竞争和错误分支仍可能增加前台开销，不能宣称日志调用完全不阻塞或必定保留全部内容。

systemd 在这个项目里解决什么问题？

面试时这样回答

网关作为前台进程运行，由 **systemd** 管理启动、退出后的重启和日志收集。服务配置还建立专用状态目录，并限制文件系统与运行权限，把程序本身的业务逻辑和进程管理分开。

串口访问需要相应设备权限，数据库目录也必须可写。部署时应把“进程启动了”“串口打开了”“Broker 连上了”分开确认；**network-online** 的启动顺序依赖不等于 MQTT 服务已经可达。

回到第一部分

T108 · systemd 部署、日志与自动重启怎样配合？

T061 · 优雅退出为什么要先停止生产者，再排空消费者？

T024 · fd、open file description、dup 与 close 有什么区别？

实现依据与展开

配置证据

服务使用 `Type=simple`、`Restart=on-failure`、`RestartSec=3`，并配置 `DynamicUser`、`dialout`、`StateDirectory`、`ProtectSystem`、`ProtectHome`、`PrivateTmp` 与 `NoNewPrivileges`。最终权限仍取决于目标系统的设备组和目录配置。

网关 `gateway.service.in:8` · 网关 `main.cpp:21`

继续追问

进程卡住不退出，**Restart=on-failure** 会自动修好吗？

不会。该策略主要在服务失败退出等情况下重启；当前没有应用向 **systemd** 定期发送 **watchdog** 心跳的实现，卡死检测需要另外设计。

当前实现边界

部署配置存在不代表已经在所有目标发行版验证，尤其需要核对设备权限与 **systemd** 版本支持。

节点采集、任务与串口

P034 项目问答

节点从复位到业务任务运行经历什么过程？

面试时这样回答

复位后先按向量表进入启动代码，准备栈、复制初始化数据并清零未初始化数据区，再进入 C 运行环境与 main。main 完成 HAL、时钟和外设配置，初始化微秒计时资源，然后建立应用队列、流缓冲和任务，最后启动调度器。

这里必须区分调度器启动前后的等待。传感器初始化期间不能假定 RTOS 已能调度；依赖 DWT 周期计数器的延时，也必须先打开计数器。任务创建和通信对象创建失败要在启动阶段发现，否则后续任务可能拿空句柄继续运行。

回到第一部分

T062 · STM32 上电后如何走到 main，再启动 FreeRTOS？

T063 · Flash、SRAM、DATA、BSS、堆和任务栈如何分布？

T064 · STM32 时钟树、APB 定时器倍频与 PWM 怎样计算？

T070 · HAL、LL 和裸寄存器怎么取舍？

T084 · FreeRTOS 的任务状态和调度链路如何理解？

T094 · 如何估算任务栈，为什么中断栈要单独看？

T100 · 忙等、任务延时、硬件计时怎样选择？

实现依据与展开

芯片与时间基准

当前目标为 STM32F103、Cortex-M3，系统时钟配置为 72 MHz；HAL 毫秒时基由 TIM4 提供，RTOS 使用 SysTick。链接脚本描述的 Flash 为 64 KiB、SRAM 为 20 KiB。

节点 startup_stm32f103xb.s:61 · 节点 main.c:116 · 节点 dwt_delay.c:7 · 节点 app.c:13 · 节点 FreeRTOSConfig.h:42

继续追问

为什么不能只看 main 就解释启动？

main 之前已经发生栈设置、数据区初始化和系统初始化；链接脚本与启动汇编决定程序在上电时能否正确到达 main。

当前实现边界

这些是源码与链接配置事实；当前没有借此给出上电到首帧的实测时间。

P035 项目问答

三个任务的优先级为什么这样安排？中断优先级又怎么算？

面试时这样回答

命令任务优先级为 3，发送任务为 2，采样任务为 1。命令任务负责消费串口输入和推进协议状态，所以优先级最高；发送任务承接已经编码好的帧；周期采样的紧迫性相对较低。但高优先级任务如果等锁或同步等传感器，仍然可能错过及时处理后续输入的机会。

NVIC 是另一套优先级体系，数值越小中断紧迫性越高，不能和任务优先级直接比较。UART 与 RX DMA 中断使用逻辑优先级 6，RTOS 可调用 API 的最高逻辑紧迫级界限为 5，因此这些 ISR 可以使用适用的 FromISR 接口。

回到第一部分

T065 · NVIC 优先级为什么数值越小越高？抢占与子优先级怎样区分？

T067 · PRIMASK、BASEPRI 与临界区有什么区别？

T084 · FreeRTOS 的任务状态和调度链路如何理解？

T085 · SysTick、PendSV、SVC 为什么分工？

T086 · FromISR API 与普通 API 有什么本质差异？

T116 · 固定优先级任务怎样做最坏响应时间分析？

实现依据与展开

编码与分组

芯片实现 4 个优先级位；当前使用全抢占位的分组。逻辑值 5 写入高 4 位后为 0x50，用于 BASEPRI 屏蔽阈值。应用不能把未移位的 5 和寄存器编码 0x50 混用。

节点 app.c:13 · 节点 FreeRTOSConfig.h:42 · 节点 dma.c:39 · 节点 usart.c:32 · 节点 stm32f1xx_it.c:87

继续追问

给命令任务再加一级优先级能解决所有超时吗？

不能。它无法抢过被屏蔽的中断，也无法消除自己等待 I²C 转换或互斥锁的时间。需要先找出响应时间由执行、阻塞还是更高优先级干扰主导。

当前实现边界

优先级分配是当前实现；最坏响应时间上界需要测量执行与阻塞时间后才能验证。

P036 项目问答

一段 UART 数据怎样从 DMA 到命令解析？

面试时这样回答

USART1 的 RX DMA 循环写入 64 字节缓冲。收到半满、满或空闲事件后，回调根据当前写入位置与上次位置计算新增区域，把新增字节复制到 `StreamBuffer`，再由命令任务取出并交给持续保留状态的帧解析器。

IDLE 表示线路暂时空闲，不表示业务帧完整。一个帧可以跨多个 DMA 事件，一个事件里也可以有多个帧，所以帧边界必须由长度、同步头和 CRC 判断。回调只搬运新增数据，完整协议处理在任务里进行。

回到第一部分

T073 · DMA 解决什么问题，为什么还需要中断和缓冲？

T074 · 循环 DMA 加 IDLE 怎样计算新数据，IDLE 是帧结束吗？

T075 · 怎样计算 DMA 与软件缓冲容量，证明不会被覆盖？

T088 · `StreamBuffer` 与 `Queue`、`MessageBuffer` 有什么区别？

T112 · 零拷贝为什么首先是缓冲所有权问题？

实现依据与展开

容量与消费

DMA 环和 `StreamBuffer` 各为 64 字节；HAL RxEvent 的 Size 按该实现表示进度位置，不能每次直接当成新增长度。命令任务一次最多读取 32 字节，无输入时最多等 100 ms。

节点 `uart_link.c:41` · 节点 `cmd_service.c:77` · 节点 `dma.c:39` · 节点 `stm32f1xx_hal_uart.c:2350`

继续追问

UART 和 DMA 两类回调会不会同时改上次位置？

它们使用相同抢占优先级，不互相嵌套，从而维持该生产侧访问的串行性；修改优先级后需要重新检查 `StreamBuffer` 单写者前提和位置更新。

当前实现边界

当前是两级缓冲之间复制的接收链，并非零拷贝；回调没有完整处理软件流缓冲写不下的情况。

P037 项目问答

64 字节接收缓冲为什么可能溢出？能算出时间预算吗？

面试时这样回答

115200、8N1 每字节在线路上占 10 bit，连续输入速率约为 11520 字节每秒，因此 64 字节大约 5.56 ms 就会被填满，半环只有约 2.78 ms。这个计算给出了连续流下的紧迫程度，不能等价于系统允许随意停顿 5.56 ms，因为已有占用和软件缓冲也要计入。

当前命令执行与解析在同一任务里，执行传感器命令可能让消费者停顿；DHT11 临界区还会推迟相关中断。若停顿期间持续有数据，DMA 可能绕回覆盖，或 `StreamBuffer` 无法接下全部新增字节。最高 69 字节的协议帧也大于单个 64 字节环，必须允许分段解析。

回到第一部分

T072 · UART 为什么无需时钟线，115200 能传多少有效数据？

T074 · 循环 DMA 加 IDLE 怎样计算新数据，IDLE 是帧结束吗？

T075 · 怎样计算 DMA 与软件缓冲容量，证明不会被覆盖？

T088 · `StreamBuffer` 与 `Queue`、`MessageBuffer` 有什么区别？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

实现依据与展开

为什么难定位

`StreamBuffer` 写入函数会返回实际接收数量，但调用处没有处理短写；DMA 环绕得太多还可能使仅靠当前位置差值无法恢复真实丢失量。应分别记录软件短写、硬件错误和解析失步。

节点 `uart_link.c:41` · 节点 `cmd_service.c:77` · 节点 `dht11.c:51` · 节点 `bh1750.c:47` · 网关 `edge_proto.h:110`

继续追问

把缓冲扩大到 256 字节是否足够？

它扩大可承受突发和停顿的范围，但不足以证明安全。需要结合最长传感器阻塞、输入突发和中断延迟计算，再做连续流压力验证。

当前实现边界

5.56 ms 是线路速率推导，不是已经测得的最大安全停顿或实际丢包时间。

P038 项目问答

串口发生 ORE 等错误后，为什么可能再也收不到数据？

面试时这样回答

错误可能使 HAL 接收状态或 DMA 流程停止，光清一个错误标志不一定恢复整个接收链。当前错误回调会更新错误计数、复位接收位置并尝试重新启动 DMA 接收，但重启 API 的返回结果还没有形成完整的成功确认与失败重试闭环。

所以排查时要分别看 USART 是否仍收到字节、DMA 计数是否变化、RxEvent 是否继续触发、StreamBuffer 是否有新增，以及任务是否仍消费。任务能定期报心跳，只说明它在运行，不能证明接收驱动已恢复。

回到第一部分

T073 · DMA 解决什么问题，为什么还需要中断和缓冲？

T076 · UART 的 ORE、FE、NE 出错后，为什么需要恢复 DMA？

T106 · 逻辑分析仪、示波器、GDB 各适合查什么？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

T115 · 错误码、异常安全与恢复重试怎样形成一致契约？

实现依据与展开

可落实的改进

把恢复请求交给明确的恢复状态机，在有限次数和时间内检查启动结果，并暴露恢复成功/失败计数；ISR 只完成必要止损和通知，避免在错误中断里无限重试。

节点 `uart_link.c:41` · 节点 `stm32f1xx_hal_uart.c:2350` · 节点 `cmd_service.c:77`

继续追问

是否直接调用 HAL_UART_Abort 就最稳妥？

需要先确认它会影响 RX、TX 中哪些状态，是否会中断另一条正常传输。恢复对象、缓冲所有权和回调完成条件必须一起设计。

当前实现边界

有限恢复状态机是改进建议；当前源码不能证明所有 UART 错误都能自动恢复。

P039 项目问答

当前串口发送是 DMA 吗？若优化，核心难点是什么？

面试时这样回答

当前发送由独立任务处理，但任务内使用 HAL_UART_Transmit 和无限等待参数，属于阻塞发送。其他任务可以被调度，不代表发送任务自身没有轮询占用 CPU；它也没有缩短串口线路上发送这些 bit 所需的时间。

合理的改造是保留发送队列，由发送任务启动 TX DMA 后阻塞等待完成。但队列取出的帧必须一直有效，直到最终 UART TC 确认最后一位已离开移位寄存器；DMA 把最后一字节搬到数据寄存器，还不是线路发送结束。还要定义启动失败、超时和取消时谁回收缓冲。

回到第一部分

T072 · UART 为什么无需时钟线，115200 能传多少有效数据？

T073 · DMA 解决什么问题，为什么还需要中断和缓冲？

T087 · 队列为什么复制数据，传指针又有什么代价？

T089 · 任务通知的计数、事件与数值语义如何选择？

T112 · 零拷贝为什么首先是缓冲所有权问题？

实现依据与展开

当前参数

TX 队列复制保存 8 个 TxFrame，帧数组容量 70 字节；入队不等待且未处理失败，发送任务空闲时最多等待 800 ms。USART1 TX DMA 对应 DMA1 Channel4，现有 RX 使用 Channel5，二者不能混为一个通道。

节点 tx_service.c:22 · 节点 tx_service.h:4 · 节点 uart_link.c:41 · 节点 dma.c:39 · 节点 stm32f1xx_hal_uart.c:2350

继续追问

DMA 发送超时能否把整条 UART 接收也重启？

不应未经分析就这样做。应先定义仅 TX 的停止与完成收敛，避免恢复发送时破坏正在运行的循环 RX DMA。

当前实现边界

TX DMA 尚未实现，也没有 CPU 占用或功耗收益的实测结果。

P040 项目问答

为什么传感器已经加锁，命令响应还会慢？

面试时这样回答

锁解决的是两个任务同时访问传感器的问题，不保证持锁时间短。当前统一的传感器互斥锁覆盖完整读取过程，BH1750 的等待也在锁内，因此周期采样与主动查询会互相等待。

即使发生优先级继承，被提升的任务如果正在阻塞等待测量，也不能凭优先级让传感器提前完成。更合理的后续设计是把设备访问交给一个服务任务，按转换时间分阶段推进，再向其他任务提供带时间戳的缓存；若命令必须要新样本，则明确异步等待与超时语义。

回到第一部分

T016 · 阻塞、非阻塞、同步、异步如何区分？

T078 · I2C 七位地址如何编码，传感器转换与读取怎样配合？

T090 · 优先级继承具体怎样工作，为什么仍不能保证快速响应？

T091 · 临界区、挂起调度器和互斥量分别适合什么？

T116 · 固定优先级任务怎样做最坏响应时间分析？

实现依据与展开

当前所有者

周期采样任务和命令任务都可能调用 `sensor_hub`；互斥锁防止总线操作交叉，但也让不同传感器共享一个较大的串行区间。

节点 `sensor_hub.c:19` · 节点 `bh1750.c:47` · 节点 `dht11.c:51` · 节点 `cmd_service.c:77` · 节点 `report_task.c:55`

继续追问

能否直接把锁挪到 `HAL_I2C_Receive` 前后？

不一定。测量模式配置与结果读取可能构成设备级事务，拆开后另一请求可能改变模式。应先定义设备状态所有权，再缩小锁范围。

当前实现边界

服务任务与缓存是建议；当前仍同步读取，不能声称已实现无阻塞采集。

P041 项目问答

BH1750 的地址、等待和总线恢复怎样解释？

面试时这样回答

传感器的 7 位地址是 0x23，而 STM32 HAL 这一接口接收的是左移后的地址形式，不能把两种表示混用。当前使用连续高分辨率模式，按默认测量时间设置把原始读数除以 1.2 转成照度，并在读取前等待测量时间。

失败后驱动尝试恢复总线并重试一次：输出时钟帮助被从设备拉住的数据线释放，再产生 STOP、重新初始化 I²C。这个方法针对部分协议中途被打断的情况；如果时钟线始终被拉低或器件供电异常，仅补几个时钟不一定有效。

回到第一部分

T077 · I2C 为什么必须开漏，上拉电阻怎么影响时序？

T078 · I2C 七位地址如何编码，传感器转换与读取怎样配合？

T079 · SDA 卡低时为什么发九个时钟，哪些故障救不了？

T100 · 忙等、任务延时、硬件计时怎样选择？

实现依据与展开

时间单位与限制

代码的 vTaskDelay(180) 单位是 Tick，在当前 1 kHz 配置下才对应约 180 ms 的标称 Tick 时长。ROHM 手册给出高分辨率测量典型 120 ms、最大 180 ms，换模式或 MTreg 后应重新核算。

节点 bh1750.c:47 · 节点 bh1750.h:8 · 节点 i2c.c:30

继续追问

为什么持续测量模式还可能等待？

当前实现按自己的命令与读取顺序保留等待；连续模式的设备能力不代表驱动已充分利用后台连续转换来返回缓存样本。

当前实现边界

恢复路径未完整检查 SCL 是否真正拉高和所有初始化返回值，因此不能保证任意总线故障都恢复成功。

P042 项目问答

DHT11 为什么使用 DWT 和临界区？代价在哪里？

面试时这样回答

DHT11 用脉冲宽度区分数据位，需要微秒级测量。当前先用毫秒等待拉低启动信号，再进入较短但仍可能持续数毫秒的临界区，使用 DWT 周期计数器判断响应和位宽；边沿等待设置了有限超时，避免正常计数器运行时永远卡住。

临界区降低了被部分中断打断而误判脉冲的机会，同时也推迟了 UART/DMA 等可屏蔽中断。这是在读数可靠性和系统响应之间的取舍。必须先启用 DWT，并让周期到时间的换算符合实际主频，否则有限超时本身也可能失效。

回到第一部分

T067 · PRIMASK、BASEPRI 与临界区有什么区别？

T080 · DHT11 如何区分 0 和 1，为什么不能用普通毫秒延时？

T091 · 临界区、挂起调度器和互斥量分别适合什么？

T100 · 忙等、任务延时、硬件计时怎样选择？

T101 · 长时间屏蔽 SysTick，会不会丢系统时间？

实现依据与展开

具体边界

启动低电平约 18 ms 的 HAL 等待在临界区外，微秒启动与逐位采集在临界区内；当前换算依赖 72 MHz。所引用 DHT11 V1.3 手册要求读取间隔大于 2 s，而应用允许 1 s 周期，主动查询也未统一限制最小间隔。

节点 dht11.c:51 · 节点 dwt_delay.c:7 · 节点 sensor_hub.c:19 · 节点 report_task.c:55

继续追问

能否宣称每次查询都返回刚刚测量的值？

不能。该手册说明读取值与前次测量有关，还有限定读取间隔。应确认实物版本，并用统一采样缓存记录更新时间，避免把收到一帧等同于即时新测量。

当前实现边界

间隔不匹配和中断延迟是当前待改进点；尚未测量真实临界区长度及其对串口的影响。

P043 项目问答

温湿度小数、无效值和采样时间怎样传给网关？

面试时这样回答

温湿度在线上按规定缩放成整数，避免直接传 C 结构体或依赖浮点布局；光照也按协议范围和舍入规则编码。读取失败时用有效性状态区分，不把一个看似正常的数值当作新样本继续上报。

当前温度编码采用无符号范围，因此驱动即使能识别某些 DHT11 版本的负号，也不能在不修改协议语义的情况下原样表示负温度。网关写入的时间主要表示接收时间，也不能直接当作传感器完成测量的精确时间。

回到第一部分

T006 · 结构体为什么不能直接当作串口协议发送？

T007 · 整数溢出、符号转换和时间回绕有哪些隐蔽问题？

T010 · 如何回答未定义行为、越界与优化后才出现的 bug？

T114 · 定点编码、单位、有效性与采样时间如何设计？

实现依据与展开

当前读数规则

温湿度小数按驱动校验后的十分位组合；负号和超出允许小数范围的数据会被拒绝。传感器无效时不发送对应有效数据帧，状态帧仍描述有效位。扩展负值应同时修改双端范围、解码和兼容策略。

节点 `sensor_hub.c:19` · 网关 `edge_proto.h:110` · 节点 `report_task.c:55` · 网关 `GatewayApp.cpp:113`

继续追问

用 0 代表读取失败可以吗？

不合适，0 可能是合法测量值。应把值、单位、缩放、有效性和时间含义独立定义，必要时再加错误原因或样本年龄。

当前实现边界

本题描述当前协议与有效性处理，不把接收时间精度当成采样同步精度。

时钟、低功耗与故障定位

P044 项目问答

为什么同时保留 TIM4 和 SysTick 两个时间基准？

面试时这样回答

两者服务的层次不同：TIM4 推进 HAL 毫秒计数，SysTick 推进 RTOS 调度时间。保留独立 HAL 时基，可以让调度器尚未启动时的 HAL 超时和初始化等待继续工作，也避免简单把 RTOS Tick 当成所有驱动接口的时间来源。

但双时基需要明确低功耗语义。当前进入 Tickless 前暂停 HAL Tick，唤醒后恢复，睡眠期间 HAL 的 uwTick 没有按实际经过时间补回；RTOS 则由移植层补偿调度 Tick。因此两边都叫“毫秒”并不意味着经过低功耗后数值仍同步。

回到第一部分

T064 · STM32 时钟树、APB 定时器倍频与 PWM 怎样计算？

T085 · SysTick、PendSV、SVC 为什么分工？

T096 · HAL Tick 与 RTOS Tick 各管什么，合并时基有什么前提？

T097 · Tickless Idle 的完整进入、唤醒和补偿过程是什么？

T100 · 忙等、任务延时、硬件计时怎样选择？

实现依据与展开

当前配置

RTOS Tick 为 1 kHz，TIM4 配置为 HAL 的 1 ms 中断，优先级为 15。DWT 微秒计数又是第三种时间来源，不应拿暂停状态不同的计数器直接互相相减。

节点 stm32f1xx_hal_timebase_tim.c:42 · 节点 FreeRTOSConfig.h:42 · 节点 port.c:455 · 节点 stm32f1xx_it.c:87

继续追问

合并为 SysTick 是否更简洁？

表面更少一个定时器，但必须覆盖调度器启动前、临界区、中断内 HAL 调用和 Tickless 期间的计时需求。先证明这些路径再合并，不能只把 HAL_IncTick 搬进 SysTick_Handler 就结束。

当前实现边界

当前实现确实保留双时基；尚未完成统一时基改造或低功耗时间一致性验证。

P045 项目问答

周期上报为什么用 `vTaskDelayUntil`？真的是严格每秒执行吗？

面试时这样回答

`vTaskDelayUntil` 按上一次计划唤醒点推算下一次时间，减少“处理耗时再加相对延时”造成的累计漂移。当前采样任务以一秒为基本检查周期，兼顾看门狗检查与上报周期判断，默认上报间隔为五分钟，命令可修改配置周期。

它保证的是计划唤醒时间的计算方式，不保证任务在那个瞬间立即运行。高优先级任务、临界区或锁等待都会造成延迟；当前逻辑每轮把累计值加 1000，并不等于直接测量真实墙上时间，所以严重超期后的行为也需要按实际循环分析。

回到第一部分

T092 · `vTaskDelay` 与 `vTaskDelayUntil` 的周期误差有什么区别？

T096 · HAL Tick 与 RTOS Tick 各管什么，合并时基有什么前提？

T116 · 固定优先级任务怎样做最坏响应时间分析？

实现依据与展开

参数范围

周期设置范围为 1 至 65535 秒。首次采样通过初始状态触发；后续的逻辑累计与硬件测量最小间隔还需要相容，尤其是 DHT11。

节点 `report_task.c:55` · 节点 `sensor_hub.c:19` · 节点 `FreeRTOSConfig.h:42`

继续追问

周期任务错过多个周期后该怎么办？

可以追赶、跳过旧周期或立即执行一次再重定相位，但应按业务明确选择。补发很多过时遥测通常没有价值，也可能进一步挤压命令处理。

当前实现边界

一秒是目标检查节奏，不是已证明的最坏调度延迟上界。

P046 项目问答

开启 **Tickless Idle** 后，节点进入了什么低功耗状态？

面试时这样回答

当前使用 Cortex-M3 移植层的 **Tickless Idle**，在空闲条件满足时延长 **SysTick** 计时并执行 **WFI**，减少周期 **Tick** 唤醒。这里主要是 **CPU Sleep**，不能直接描述为已经进入 **STM32 Stop** 或 **Standby**；后两者涉及额外的时钟、外设和唤醒恢复流程。

能睡多久由最近一个任务截止时间、外部中断和计时器范围共同决定。命令任务的周期等待约 **100 ms**，已经限制了很多空闲区间；**SysTick** 还只有 **24** 位计数器。睡前暂停 **TIM4** 的 **HAL Tick**，避免它每毫秒把 **CPU** 唤醒，醒来再恢复。

回到第一部分

T096 · **HAL Tick** 与 **RTOS Tick** 各管什么，合并时基有什么前提？

T097 · **Tickless Idle** 的完整进入、唤醒和补偿过程是什么？

T098 · 硬件计数器位宽怎样限制一次休眠时长？

T099 · **Sleep**、**Stop**、**Standby** 的保持与唤醒机制有什么区别？

实现依据与展开

理论数量级

在 **SysTick** 使用 **72 MHz** 计数时，**24** 位最大周期约为 **233 ms**；这是计数范围推导，真实抑制 **Tick** 数还受移植实现的取整、补偿和更早唤醒约束。

节点 `FreeRTOSConfig.h:42` · 节点 `port.c:455` · 节点 `report_task.c:55` · 节点 `cmd_service.c:77` · 节点 `stm32f1xx_hal_timebase_tim.c:42`

继续追问

串口 **DMA** 在这种状态下还能收吗？

需要按实际 **Sleep** 时的总线和外设时钟配置确认。不能把浅睡眠下能工作推导为 **Stop** 模式下也保持同样行为。

当前实现边界

已有 Tickless 配置与钩子，不代表已测得整板电流下降比例；没有提供板级功耗数据。

P047 项目问答

看门狗怎样判断系统健康？为什么任务有心跳还可能丢串口？

面试时这样回答

三个业务任务分别设置自己的健康位，采样任务周期检查这些位都出现后才喂 IWDG，并清除本轮标记。这样比任何一个任务无条件喂狗更容易发现某个任务长时间卡住，但健康位只能代表该任务到达了检查点。

例如命令任务还能超时醒来报心跳，UART RX DMA 却可能已经停止，因此任务活着并不等于输入链路健康。要覆盖这种故障，需要把驱动状态、最后输入进展和恢复结果纳入诊断，同时区分“正常没有数据”和“有输入却不前进”。

回到第一部分

T052 · 心跳、健康检查与有效进展分别能证明什么？

T095 · 软件定时器和事件组有哪些容易忽略的上下文问题？

T102 · IWDG 与 WWDG 怎样工作，超时与窗口如何计算？

T103 · 怎样设计能检查有效进展的多任务看门狗？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

超时与竞态

当前 IWDG 预分频为 64、重装载值为 1250；按 LSI 标称 40 kHz，超时约为 $(1250+1) \times 64 / 40000 \approx 2.0016$ s，实际受 LSI 偏差影响。读取事件位、喂狗、清位并非一个原子事务，其间新设置的位可能被清掉。

节点 watchdog.c:19 · 节点 report_task.c:55 · 节点 cmd_service.c:77 · 节点 tx_service.c:22 · 节点 iwdg.c:41

继续追问

调试时为什么不复位，脱离调试器却复位？

工程配置了调试暂停时冻结 IWDG 等行为，因此断点环境与独立运行条件不同。还要读取并保存复位原因，区分看门狗、上电和软件复位。

当前实现边界

该健康检查不是对业务正确性的完整证明，当前也没有故障注入得到的看门狗覆盖率。

P048 项目问答

节点的栈和堆预算怎么讲？发生 **HardFault** 如何留现场？

面试时这样回答

要把链接期 RAM 占用、RTOS 堆和各任务栈一起算。任务创建的栈深度单位是 `StackType_t` 元素，在这个 32 位移植上一个元素为 4 字节；只报“384 字节栈”会把实际分配说错。堆里除任务栈外还可能有 TCB、队列和事件对象。

当前使用 `heap_4`，总堆为 10 KiB，芯片 SRAM 为 20 KiB，因此需要结合链接 `map`、最低剩余堆和任务栈高水位判断余量。**HardFault** 当前停在死循环，若要定位现场，应额外保存异常栈帧、`EXC_RETURN`、故障状态寄存器和复位原因，再通过可靠途径导出。

回到第一部分

T063 · Flash、SRAM、DATA、BSS、堆和任务栈如何分布？

T068 · **HardFault** 应当怎样定位，为什么不能只在里面死循环？

T093 · FreeRTOS `heap_1` 到 `heap_5` 的分配策略与适用条件是什么？

T094 · 如何估算任务栈，为什么中断栈要单独看？

T103 · 怎样设计能检查有效进展的多任务看门狗？

实现依据与展开

已配置的栈

命令与采样任务各 384 个字，发送任务 256 个字；Idle 最小栈配置 128，定时器任务配置 256。是否实例化、控制结构和堆块开销也要纳入实际构建，而不能只把几个数字相加当最终 RAM 使用量。

节点 `app.c:13` · 节点 `FreeRTOSConfig.h:42` · 节点 `heap_4.c:173` · 节点 `STM32F103XX_FLASH.Id:56` · 节点 `stm32f1xx_it.c:87`

继续追问

高水位很大就能证明永远不溢出吗？

不能，它只反映已经走过的执行路径；中断嵌套、错误处理和格式化等最坏路径还需覆盖。当前 `malloc` 失败与栈溢出钩子也未全部启用。

当前实现边界

现场采集与更完整监测是建议；当前没有伪造任何板上高水位或最大堆使用数据。

P049 项目问答

用户说“命令经常超时”，你会沿哪条链定位？

面试时这样回答

先用同一请求身份串起各阶段：MQTT 收到、GatewayCore 接纳与解释、进入串口命令队列、取得窗口、串口完整提交、节点接收确认、节点执行、结果返回。超时只是最终现象，必须先定位最后一个已经确认发生的事件，才知道该查网络、排队、线路还是执行。

如果始终没有接收确认，重点看发送进展、帧校验、DMA 与软件缓冲丢失；已经确认但没有结果，则继续查有序执行的前序缺口、传感器阻塞和结果发送。网关侧还要分别检查串口回调占用、Core 排队和 control 重载等待；SR 定时扫描延迟与业务发布延迟属于不同线程路径。

回到第一部分

T030 · timerfd 怎样报告超时，时钟和定时精度如何选择？

T039 · 传输确认、按序交付与业务完成为什么不是同一状态？

T041 · 怎样设计 RTO、总截止时间与有限重试？

T104 · CPU 很高、内存增长、延迟变大分别怎么定位？

T105 · 一个请求超时，如何分层定位并判断远端是否已执行？

T106 · 逻辑分析仪、示波器、GDB 各适合查什么？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

建议记录的证据

用单调时钟记录每阶段时间，携带会话、序号、业务请求 ID 与错误原因；同时采集队列深度、最老项年龄和溢出计数。先建立一条正常基线，再注入单点丢帧或慢传感器验证定位是否成立。

网关 MqttClient.cpp:21 · 网关 GatewayApp.cpp:113 · 网关 CommandTracker.cpp:78 · 网关 NodeLink.cpp:92 · 节点 uart_link.c:41 · 节点 cmd_service.c:77 · 节点 sensor_hub.c:19

继续追问

直接把 500 ms 超时改成 2 s 能解决吗？

可能减少误报，也可能掩盖阻塞并延长失败发现时间。应先测真实 RTT 与执行等待，并区分链路接收期限和业务结果期限，再调整参数。

当前实现边界

以上是可执行的定位方法；源码中尚未具备这里列出的全部关联追踪点，不能把方案当现有监控结果。

P050 项目问答

哪些结论可以靠主机测试证明，哪些必须上板？

面试时这样回答

主机测试适合证明帧解析和状态机对一组确定输入的行为，例如分片、CRC 错误、重复、乱序、丢确认、超时和会话切换；用假时钟还能精确构造截止时间前后的事件。编译与链接检查则能发现接口、目标架构和静态资源配置问题。

DMA 时序、真实中断延迟、DHT 脉冲、I²C 电气恢复、看门狗复位以及整板功耗，仍需要硬件环境。二者应该形成递进证据：先把纯逻辑做成可重复实验，再用逻辑分析仪、调试器和故障注入验证硬件边界。

回到第一部分

T045 · 如何验证协议状态机，而不只测试正常收发？

T106 · 逻辑分析仪、示波器、GDB 各适合查什么？

T107 · 测试应该分哪些层，怎样覆盖时序与并发边界？

T109 · 怎样设计一个可移植、可测试的驱动或协议模块？

实现依据与展开

如何报告验证

明确测试的源码版本、输入场景、断言、执行环境和实际输出；构建通过不等于功能通过，若未本次运行就不要把历史结果写成本次验证。仓库中已有测试资产也不自动证明个人独立完成了它们。

网关 edge_frame.c:101 · 网关 edge_src.c:62 · 网关 CommandTracker.cpp:78 · 网关 CMakeLists.txt:2 · 节点 freertos.cmake:17

继续追问

只测试一整帧一次送入够吗？

不够。串口没有业务帧边界，应枚举不同切分、连续多帧、噪声前缀、错误长度与恢复路径，并检查解析器内存边界和后续重同步。

当前实现边界

本文修订只验证文档、引用和源码基线；不冒充一次新的固件上板或业务压力测试。

P051 项目问答

如果继续优化项目，你会先做什么？

面试时这样回答

我会先补可观测性和有限失败处理：记录队列拒绝与 **StreamBuffer** 短写，检查 **DMA** 重启结果，限制可能长期阻塞的等待，并把数据库写入结果传回上层。这些改动先让丢失和失败可发现、可解释，也为后面的性能优化提供基线。

然后再处理最长的阻塞链，把慢传感器访问改成有明确样本语义的服务或缓存，并评估 **TX DMA**。零拷贝放在测得复制确实成为瓶颈之后，因为几十字节复制换来的所有权简单性可能更有价值。

回到第一部分

T035 · 事件循环如何同时保证就绪处理、公平性与退出？

T075 · 怎样计算 **DMA** 与软件缓冲容量，证明不会被覆盖？

T086 · **FromISR** API 与普通 API 有什么本质差异？

T102 · **IWDG** 与 **WWDG** 怎样工作，超时与窗口如何计算？

T111 · 有界队列、背压与丢弃策略如何从负载推导？

T112 · 零拷贝为什么首先是缓冲所有权问题？

T115 · 错误码、异常安全与恢复重试怎样形成一致契约？

实现依据与展开

如何验证收益

针对每一项给出原始问题、修改前后同负载数据和副作用：看最坏命令等待、丢帧计数、任务 **CPU** 占用、**RAM** 峰值与错误恢复时间。**DMA** 收益应明确是减少轮询时间，不是提高 **115200** 线路带宽。

节点 `uart_link.c:41` · 节点 `tx_service.c:22` · 节点 `sensor_hub.c:19` · 节点 `watchdog.c:19` · 网关 `Database.h:110` · 网关 `GatewayApp.cpp:113`

继续追问

为什么不先换更快 **MCU**？

若根因是无限等待、缓冲策略或完成语义错误，更快 **MCU** 只能部分掩盖问题。资源升级可以是方案，但应先定位约束来源。

当前实现边界

本题是后续工作的优先级建议，不把尚未实现的方案写入已完成的个人成果。

P052 项目问答

这个项目最值得展开的技术难点是什么？怎样把它讲清楚？

面试时这样回答

一个值得展开的难点是：在丢帧、重传和乱序下，如何既推动命令完成，又不把重复数据当成新的执行。可以沿一条请求讲清楚：网关分配会话与序号，节点接纳后确认，按序执行并缓存结果，网关处理结果后再确认结果，重复帧则命中原状态而不是重新执行。

紧接着讲约束就容易理解：缓存有限、会话有生命周期、超时不能证明远端没执行，掉电也会丢失内存状态。这样每个确认和定时器都有明确目的，面试官可以沿“丢的是哪一帧”继续追问，而不必听一串类名和函数名。

回到第一部分

T038 · 停止等待、GBN 与 SR 怎样工作，窗口上限如何推导？

T039 · 传输确认、按序交付与业务完成为什么不是同一状态？

T040 · 会话号与序号为什么缺一不可？

T041 · 怎样设计 RTO、总截止时间与有限重试？

T043 · 去重缓存满了、过期了或结果 ACK 丢了会怎样？

T113 · 状态机怎样把事件、超时和错误恢复组织清楚？

实现依据与展开

可以展开的两个例子

结果帧先于接收确认到达时，网关可把结果视为更强的接收证据；前序序号缺口长期无法恢复时，后续有序执行不能假装成功跳过，因此需要明确会话失败与恢复策略。分别对应 **tracker** 和共享 **SR** 状态机。

网关 `CommandTracker.cpp:78` · 网关 `edge_sr.c:62` · 网关 `NodeLink.cpp:92` · 节点 `uart_link.c:41` · 节点 `tx_service.c:22`

继续追问

怎样说明自己做了什么？

只陈述自己实际负责的修改、定位和验证，并拿对应提交或记录说明。当前代码能说明实现机制，不能自动证明全部代码、前端和测试都由同一个人独立完成。

当前实现边界

本文提供可核对的技术叙述；个人贡献范围应以真实工作记录为准。

FreeRTOS 移植与启动验收

P053 项目问答

这个节点具体怎样手动移植 FreeRTOS?

面试时这样回答

这个节点基于 STM32F103 的 CubeMX HAL 工程，手动接入 FreeRTOS 原生内核。编译时选 GCC/Cortex-M3 的 port.c、heap_4.c 和实际用到的任务、队列、定时器、事件组、流缓冲源码，把内核头、port 头和 FreeRTOSConfig.h 加入搜索路径；配置 72 MHz CPU、1 kHz Tick、10 KB RTOS 堆和任务优先级，然后由 app_start 创建对象与三个任务并启动调度器。

运行时关键是把启动向量表的 SVC、PendSV、SysTick 交给 FreeRTOS port，禁用 CubeMX 生成的同名空中断函数。USART1 与 RX DMA 的中断优先级设为 6，满足 FromISR 的门槛 5；HAL 继续用 TIM4 管自己的毫秒时基，RTOS 用 SysTick。源码能证明这些接线和参数，是否满足时序、栈余量及板上功耗还要分别验证。

移植分成构建接线和上电运行两条核查路径

构建接线



上电运行



回到第一部分

T065 · NVIC 优先级为什么数值越小越高？抢占与子优先级怎样区分？

T086 · FromISR API 与普通 API 有什么本质差异？

T117 · 把 FreeRTOS 移植到 Cortex-M3，源码和编译选项怎样选？

T118 · SVC、PendSV、SysTick 怎样接入向量表并启动第一个任务？

T119 · FreeRTOSConfig.h 的时钟、堆栈和中断优先级怎样计算？

实现依据与展开

芯片与构建选择

目标宏为 STM32F103xB，编译选项为 -mcpu=cortex-m3，链接脚本分配 64 KiB Flash、20 KiB SRAM。CMake 的 freertos.cmake 仅编入 tasks/list/queue/timers/event_groups/stream_buffer、GCC/ARM_CM3/port.c 与 heap_4.c；内核 include、port 目录和 Core/Inc 通过 freertos_if 传给应用层。根目录的 FreeRTOS-Kernel-main 不是当前固件的编译来源。

配置参数

configCPU_CLOCK_HZ=72000000，configTICK_RATE_HZ=1000，configMAX_PRIORITIES=7，configMINIMAL_STACK_SIZE=128 个 32 位栈元素；动态分配开启、静态分配关闭、configTOTAL_HEAP_SIZE=10 KiB。软件定时器任务优先级 2、栈深 256 元素、命令队列长 10。configUSE_TICKLESS_IDLE=1，睡眠钩子暂停和恢复 TIM4 更新中断。

异常接管

启动文件向量表使用 SVC_Handler、PendSV_Handler、SysTick_Handler；FreeRTOSConfig.h 将 port 的 vPortSVCHandler、xPortPendSVHandler、xPortSysTickHandler 映射到这些名字，并开启 configCHECK_HANDLER_INSTALLATION=1。stm32f1xx_it.c 中 CubeMX 的三个空桩被 #if 0 禁用。启动调度器后，SVC 恢复首任务，SysTick 推进 tick，PendSV 完成切换。

中断优先级

CMSIS 定义 4 个优先级位，NVIC 使用全抢占位分组。最低逻辑优先级 15 编码为 0xF0，调用内核的最高紧迫逻辑级 5 编码为 0x50；USART1 和 DMA1_Channel5 配为 6。串口接收回调调用 xStreamBufferSendFromISR，并用 portYIELD_FROM_ISR 请求必要的切换。

双时基与启动顺序

main 先 HAL_Init、配置 72 MHz 时钟，再初始化 GPIO、DMA、USART1、I2C1、IWDG；HAL_InitTick 被 TIM4 实现覆盖。app_start 先启用 DWT，再创建互斥量、发送队列、接收 StreamBuffer、看门狗事件组，最后创建 Cmd 3、Trans 2、Sample 1 三个任务并调用 vTaskStartScheduler。RX DMA 在 Cmd 任务实际运行后启动。

栈与版本

Cmd、Sample 各为 384 个栈元素即 1536 B，Trans 为 256 个即 1024 B；Idle 与软件定时器任务还要分别分配栈和 TCB。链接脚本的 _Min_Heap_Size 与 _Min_Stack_Size 不等于 FreeRTOS 的堆和任务栈。工程内核是开发分支快照，版本文件未给出精确提交，不能称为已核实的正式发行版。

节点 freertos.cmake:17 · 节点 FreeRTOSConfig.h:42 · 节点 port.c:455 · 节点 startup_stm32f103xb.s:61 · 节点 stm32f1xx_it.c:87 · 节点 main.c:116 · 节点 stm32f1xx_hal_timebase_tim.c:42 · 节点 app.c:13 · 节点 STM32F103XX_FLASH.ld:56 · 节点 N6_freertos.ioc:60 · 节点 stm32f1xx_hal_conf.h:133

继续追问

为什么 HAL 配置里的 USE_RTOS 仍是 0?

它是这版 STM32 HAL 自身的编译开关，不是 FreeRTOS 总开关。当前 HAL 头文件要求 USE_RTOS 为 0；FreeRTOS 是否接入由 CMake、FreeRTOSConfig.h、异常入口和 vTaskStartScheduler 决定。

重新用 CubeMX 生成代码，哪几处最可能坏？

重点检查 SVC/PendSV/SysTick 空桩是否被重新加入、TIM4 HAL timebase 是否仍保留、NVIC 分组与 USART/DMA 优先级是否变化，以及自定义 CMake 接线是否还在。

当前实现边界

这些是当前工作区代码和构建配置可核对的实现，不据此推断移植由某个人独立完成，也不把编译通过说成已做板上时序与功耗验证。

P054 项目问答

移植后若任务不运行或串口接收卡住，怎样逐层排查并验收？

面试时这样回答

我会按启动边界逐层排查：先看 ELF 是否链接了唯一的 SVC、PendSV、SysTick 实现，再看 app_start 是否因内核对象或任务创建失败进入 Error_Handler，之后确认首任务运行、SysTick 递增、PendSV 能切换。这样能把异常接线、10 KB 堆不足、任务栈和时钟参数不一致分开定位。

调度跑起来后，再检查 USART1、DMA1_Channel5 的逻辑优先级是否保持 6，接收回调的 FromISR 写入是否真正唤醒 Cmd 任务；最后单独验证 TIM4 HAL 超时、Tickless 的 WFI 唤醒以及睡眠后的时间语义。主机编译与符号检查只能证明前段，响应、栈余量和功耗需要在板上观察或测量。

回到第一部分

T071 · 如果增加 Bootloader/OTA，怎样设计启动与回滚？

T087 · 队列为什么复制数据，传指针又有什么代价？

T095 · 软件定时器和事件组有哪些容易忽略的上下文问题？

T117 · 把 FreeRTOS 移植到 Cortex-M3，源码和编译选项怎样选？

T118 · SVC、PendSV、SysTick 怎样接入向量表并启动第一个任务？

T119 · FreeRTOSConfig.h 的时钟、堆栈和中断优先级怎样计算？

T120 · HAL 与 FreeRTOS 使用不同 Tick 时，怎样实现 Tickless 并验证移植？

实现依据与展开

构建与符号层

先检查 freertos.cmake 的 GCC/ARM_CM3/port.c 和 heap_4.c 是否各编一次，再用 nm 或 map 确认向量表的三个 Handler 来自 port.c；若仍落到启动文件的 Default_Handler 或 CubeMX 空桩，任务不会按预期调度。configCHECK_HANDLER_INSTALLATION=1 会帮助发现 SVC/PendSV 接线错误，但不能替代完整验证。

创建与调度层

app_start 对传感器互斥量、TX 队列、RX StreamBuffer、看门狗事件组及三个 xTaskCreate 逐项检查，失败进入 Error_Handler。启动后观察 Cmd、Trans、Sample 任务是否运行和阻塞唤醒，并用栈高水位、剩余堆及故障寄存器判断是否有任务栈或内存问题；当前 configCHECK_FOR_STACK_OVERFLOW=0，不能声称已开启溢出钩子。

中断与接收层

DMA1_Channel5 和 USART1 IRQ 均为逻辑优先级 6，大于等于系统调用阈值 5；HAL 回调把 DMA 新增字节写入 StreamBuffer，然后由 Cmd 任务消费。若任务还在打卡但没有命令输入，分别看 UART 错误计数、DMA 计数、RxEvent、StreamBuffer 短写和接收重启返回值，不能只凭任务存活判断接收链已恢复。

双时基与低功耗层

TIM4 的更新中断推进 HAL_IncTick，port 的 SysTick 推进任务 Tick。Tickless 睡前 HAL_SuspendTick、醒后 HAL_ResumeTick 只暂停和恢复 TIM4 更新中断；HAL 的 uwTick 没有在这段睡眠中按实时时间补偿。验收应记录唤醒源、实际驻留时间、HAL 超时与 RTOS 延时，若声称节能还需板上电流测量。

节点 freertos.cmake:17 · 节点 FreeRTOSConfig.h:42 · 节点 port.c:455 · 节点 startup_stm32f103xb.s:61 · 节点 stm32f1xx_it.c:87 · 节点 stm32f1xx_hal_timebase_tim.c:42 · 节点 app.c:13 · 节点 uart_link.c:41 · 节点 dma.c:39 · 节点 usart.c:32 · 节点 heap_4.c:173

继续追问

看到 SysTick_Handler 符号就能证明 1 ms tick 正常吗？

不能。还要确认实际系统时钟与 configCPU_CLOCK_HZ 一致、port 设置的重装载值正确，以及 tick 与任务延时能在板上观察到。Tickless 开启时也不能预期每毫秒都有中断。

为什么串口无回应不一定是 FreeRTOS 调度故障？

可能是 RX DMA 已停、StreamBuffer 写不下、Cmd 任务忙于同步传感器读取、TX 队列满或发送任务阻塞。需要沿 ISR 到任务、队列到 UART 的链路定位，不能把所有超时归因于移植层。

当前实现边界

本题是基于现有源码给出的分层诊断和验收路径；没有对应的板上测量记录，不把计划中的检查说成已经完成。